



Clasificación y Extracción de Entidades Nombradas (NER) en Noticias de Cumplimiento Normativo Corporativo Mediante Modelos de Lenguaje Grande Ejecutados Localmente con Soberanía de Datos

Eduardo Mauricio Ahumada Gallardo

Austranet — Departamento de Informática, Universidad Técnica Federico Santa María, Valparaíso, Chile
eahumada@gmail.com

RESUMEN

Las instituciones sujetas a regulaciones AML/KYC deben vigilar grandes volúmenes de noticias no estructuradas buscando entidades de riesgo. Hacerlo manualmente no escala y delegarlo en APIs en la nube expone información sensible a terceros. Este trabajo diseña, implementa y evalúa un sistema soberano de reconocimiento de entidades nombradas (NER) con modelos de lenguaje grande de código abierto ejecutados en local mediante Ollama sobre Apple Silicon, sobre una arquitectura pub/sub multihilo con concurrencia adaptativa (AIMD) y capa Factory/Facade. La validación comparó trece modelos sobre 120 artículos reales en español y 30 del dominio; contrastó la extracción directa con la generación aumentada por recuperación (RAG) contextual y midió las diferencias con ANOVA y Tukey HSD. El beneficio del RAG resulta inversamente proporcional a la capacidad del modelo: solo es significativo en los dos más débiles (+14,5 y +10,8 puntos de F1) y es nulo o adverso en los mayores. Redactar el prompt en español e incorporar ejemplos *few-shot* aporta +10,4 puntos sobre el baseline en inglés, mejora que ninguno de ambos factores logra por separado. El mejor modelo alcanza 80,57 % de F1 sin extracciones fallidas en el corpus del dominio, reduce el costo unitario de revisión y preserva la confidencialidad.

Palabras clave: Reconocimiento de Entidades Nombradas (NER), Modelos de Lenguaje Grande (LLM), Cumplimiento Normativo (AML/KYC), Soberanía de Datos, Generación Aumentada por Recuperación (RAG).

ABSTRACT

Financial institutions subject to AML/KYC regulations must monitor large volumes of unstructured news for risk entities. Doing so manually does not scale, and delegating it to cloud APIs exposes sensitive information to third parties. This work designs, implements and evaluates a sovereign Named Entity Recognition (NER) system using open-source Large Language Models executed locally through Ollama on Apple Silicon hardware, on a multithreaded pub/sub architecture with adaptive concurrency control (AIMD) and a Factory/Facade layer. Validation compared thirteen models on 120 real Spanish-language articles and 30 domain ones; contrasted direct extraction with contextual retrieval-augmented generation (RAG) and measured the differences with ANOVA and Tukey HSD. The benefit of RAG is inversely proportional to model capacity: it is significant only in the two weakest models (+14.5 and +10.8 F1 points) and is null or adverse in the larger ones. Writing the prompt in Spanish and adding *few-shot* examples yields +10.4 points over the English baseline, an improvement neither factor achieves alone. The best model reaches 80.57 % F1 with no failed extractions on the domain corpus, lowers the unit cost of review and preserves confidentiality.

Keywords: Named Entity Recognition (NER), Large Language Models (LLM), Regulatory Compliance (AML/KYC), Data Sovereignty, Retrieval-Augmented Generation (RAG).

ÍNDICE DE CONTENIDOS

Introducción
Marco Teórico y Estado del Arte
Descripción del Sistema Propuesto
Diseño Experimental
Resultados Experimentales
Discusión



Conclusiones y Trabajo Futuro
Referencias Bibliográficas
Anexos

1 INTRODUCCIÓN

1.1 Contexto y Motivación

Las instituciones financieras operan bajo un marco regulatorio estricto que les obliga a identificar y gestionar entidades de riesgo en tiempo real. Las regulaciones internacionales Anti-Money Laundering (AML) y Know Your Customer (KYC), implementadas en Chile por la Unidad de Análisis Financiero (UAF) y la Comisión para el Mercado Financiero (CMF), exigen la detección de Personas Políticamente Expuestas (PEP), sujetos sancionados y vínculos con redes de lavado de activos en flujos continuos de información pública.

El proceso actual en instituciones como Austranet implica la revisión manual de cientos de artículos periodísticos diarios por analistas especializados, un proceso cuyo costo se estima en USD 8.75 por artículo analizado. A nivel global, según Verified Market Research el mercado de RegTech se valoró en USD 15,68 mil millones en 2020 y se proyecta que alcance USD 87,17 mil millones hacia 2028, con una CAGR del 23,92 % durante 2021-2028, evidenciando la urgencia de soluciones automatizadas y escalables (Verified Market Research, 2022).

1.2 Planteamiento del Problema

El problema es de naturaleza técnico-operativa: extraer automáticamente entidades nombradas —personas y organizaciones— desde noticias no estructuradas en español, con precisión suficiente para sostener una decisión de cumplimiento, en un dominio donde la ambigüedad referencial es la norma y sin que el texto abandone la infraestructura de la institución.

Ninguna de las alternativas disponibles satisface simultáneamente esas condiciones. La **revisión manual** ofrece la máxima precisión, pero su coste crece linealmente con el volumen y no escala ante un flujo continuo de noticias. Los **sistemas basados en reglas y expresiones regulares** resultan frágiles ante la variación morfológica del español, donde un mismo nombre admite múltiples formas según la posición, la partícula y la acentuación. Las **APIs comerciales en la nube** aportan la capacidad necesaria, pero transfieren texto que puede contener información de clientes a un tercero, lo que en una entidad regulada obliga a suscribir acuerdos de tratamiento de datos y amplía la superficie de exposición; su coste, además, se vuelve prohibitivo al procesar por lotes. Los **modelos supervisados** del tipo BERT-NER alcanzan el mejor desempeño publicado, pero requieren miles de ejemplos etiquetados en el dominio objetivo, inexistentes en español para cumplimiento financiero.

La brecha que este trabajo aborda se sitúa precisamente en esa intersección vacía: obtener un desempeño aprovechable **sin datos etiquetados del dominio y sin ceder los datos a un tercero**.

1.3 Hipótesis de Trabajo

Hipótesis: Es viable implementar un sistema soberano de extracción y clasificación de entidades financieras para cumplimiento corporativo (AML/KYC) utilizando modelos de lenguaje de código abierto de escala media-grande (8B–31B parámetros) ejecutados localmente, alcanzando un desempeño competitivo en español (F1-Score $\geq 70\%$) mediante técnicas sistemáticas de prompt engineering y few-shot learning, eliminando la fuga de datos confidenciales y reduciendo los costos operativos en más del 60%.

Variables independientes: (1) Modelo LLM seleccionado; (2) Estrategia de prompt (zero-shot/few-shot, inglés/español). **Variables dependientes:** (1) F1-Score por tipo de entidad; (2) Tasa de alucinaciones; (3) Latencia de procesamiento; (4) Consumo de VRAM.

1.4 Objetivos

Objetivo General: Diseñar, implementar y validar un sistema soberano de extracción de entidades nombradas para cumplimiento normativo (AML/KYC) basado en LLMs de código abierto ejecutados localmente.

Objetivos Específicos:

Diseñar e implementar una arquitectura pub/sub multithreading con control adaptativo de concurrencia para la ejecución segura de LLMs de gran escala en hardware Apple Silicon.

Evaluar y comparar el desempeño de modelos de lenguaje de código abierto generativos (familias Gemma, Llama, DeepSeek, Qwen, Mistral, GPT-OSS, Nemotron) en la tarea de NER sobre corpus de sanciones financieras reales en español: **12 modelos** en el benchmark exploratorio N=15 (§5.1) y **13 modelos** en el estudio principal N=120 con KB RAG (§5.3.5).

Ejecutar una comparación sistemática de cuatro configuraciones de prompt (zero-shot/few-shot × inglés/español) —un diseño factorial 2×2, que la literatura anglosajona de aprendizaje automático denomina *ablation study*— para cuantificar el impacto de la localización lingüística y el aprendizaje en contexto.

Validar estadísticamente los resultados mediante ANOVA de una vía y pruebas post-hoc de Tukey HSD ($\alpha=0.05$) sobre un corpus estadísticamente significativo ($N\geq 30$).

Demostrar una reducción de costos operativos del 60–80% respecto a la revisión manual, manteniendo una tasa de alucinaciones inferior al 5%.

1.5 Enfoque de Solución y Metodología de Validación

La solución adoptada consiste en operar modelos de lenguaje generativos de código abierto **enteramente sobre infraestructura propia**, describiendo la tarea de extracción en el propio *prompt* en lugar de ajustar los pesos del modelo. Esta elección resuelve de raíz las dos restricciones del problema —no exige corpus etiquetado y no expone el texto—, a cambio de asumir dos inconvenientes que el sistema debe gestionar: una salida no estructurada por construcción, que se fuerza a un esquema verificable, y un riesgo de alucinación que debe medirse explícitamente. El capítulo 2 revisa las alternativas disponibles y el capítulo 3 justifica cada decisión frente a ellas.

La validación sigue una estrategia empírica en tres etapas. Primero se establece una **línea base comparativa** ejecutando el conjunto de modelos candidatos sobre un corpus anotado, con el fin de acotar el espacio de opciones viables. Después se aísla el efecto de las variables de *prompt* mediante un diseño factorial que cruza idioma y presencia de ejemplos, evaluando las cuatro combinaciones sobre el mismo modelo y corpus. Finalmente se contrasta la extracción directa frente a la aumentada por recuperación sobre un corpus ampliado, y se determina mediante **ANOVA de una vía y pruebas post-hoc de Tukey HSD** si las diferencias observadas son estadísticamente significativas o atribuibles a la variabilidad entre artículos. Todas las corridas quedan definidas por un fichero de configuración reproducible, así que cualquier resultado del informe pueda rehacerse a partir de los artefactos publicados.

1.6 Estructura del Documento

El capítulo 2 revisa las familias de técnicas aplicables al problema —desde los sistemas basados en reglas hasta los modelos generativos—, las estrategias de aumento por recuperación y las alternativas de ejecución local, y cierra fijando los criterios de selección. El capítulo 3 describe el sistema propuesto y justifica cada decisión de diseño frente a esos criterios. El capítulo 4 detalla el diseño experimental: corpus, modelos, configuraciones de *prompt*, métricas e infraestructura. El capítulo 5 presenta los resultados de los tres experimentos y el capítulo 6 los discute, con especial atención a la contribución metodológica sobre qué información conviene recuperar. El capítulo 7 recoge las conclusiones y las líneas de trabajo futuro. Los anexos reúnen el material de reproducción: estructura del repositorio, *prompts* completos, configuración del entorno y el análisis detallado del defecto de codificación del corpus.

2 MARCO TEÓRICO Y ESTADO DEL ARTE

Este capítulo revisa las familias de técnicas disponibles para resolver el problema planteado y establece los criterios con los que, en el capítulo 3, se selecciona una de ellas. El recorrido no pretende ser exhaustivo sino comparativo: interesa entender qué exige cada alternativa, qué garantiza y en qué condiciones deja de ser aplicable al caso de estudio, caracterizado por la ausencia de corpus etiquetados en español para el dominio de cumplimiento y por la obligación de no exponer los datos a terceros.

2.1 El problema y las familias de técnicas disponibles

El Reconocimiento de Entidades Nombradas (NER) es una subtarea del Procesamiento de Lenguaje Natural que consiste en localizar fragmentos de texto y clasificarlos en categorías semánticas predefinidas. En cumplimiento normativo las categorías relevantes son **Personas** (PER), **Organizaciones** (ORG) y **Ubicaciones** (LOC), pues son las que permiten cotejar una noticia contra listas de sanciones y de personas expuestas políticamente [19].

Formalmente se plantea como un problema de etiquetado de secuencias: dado un texto segmentado en tokens, se asigna a cada uno una etiqueta según el esquema IOB2, que distingue el inicio de una entidad (*Beginning*), su continuación (*Inside*) y el texto ajeno a toda entidad (*Outside*). Esta formulación, heredada de la tarea compartida CoNLL-2002 [12], es la que fija el criterio de evaluación: una entidad se considera correctamente extraída solo si coinciden a la vez sus límites y su categoría.

La dificultad del dominio no proviene de la definición de la tarea sino de tres rasgos del material periodístico financiero. Primero, la **ambigüedad referencial**: un mismo token puede designar una persona o una organización según el contexto —«Santander» es tanto un apellido como un banco y una ciudad—. Segundo, la **variación morfológica del español**, con nombres compuestos, partículas («de», «del», «y») y tildes que fragmentan la coincidencia exacta. Tercero, la **escasez de datos etiquetados**: no existe un corpus público en español anotado para el dominio AML/KYC, lo que descarta de entrada cualquier técnica que dependa de un volumen sustancial de ejemplos supervisados.

Las aproximaciones al problema pueden ordenarse por el tipo de conocimiento que requieren y por el coste de adaptarlas a un dominio nuevo.

Los **sistemas basados en reglas y diccionarios** (*gazetteers*) identifican entidades por coincidencia contra catálogos y por patrones léxicos escritos a mano. Son transparentes, deterministas y no requieren entrenamiento, pero su cobertura se limita a lo enumerado: fracasan ante nombres nuevos, que es precisamente el caso de interés en la detección temprana de riesgo.

Los **Campos Aleatorios Condicionales (CRF)** [17], [10] modelan la secuencia de etiquetas como un campo probabilístico condicionado al texto, capturando dependencias entre etiquetas contiguas. Superan a las reglas en generalización, pero dependen de ingeniería manual de rasgos y de un corpus anotado del orden de miles de oraciones.

Las **arquitecturas neuronales BiLSTM-CRF** sustituyen los rasgos manuales por representaciones aprendidas, eliminando gran parte del trabajo de ingeniería a cambio de un requisito de datos aún mayor.

Los **codificadores Transformer pre-entrenados** —BERT [2] y sus variantes multilingües como XLM-R— constituyen el estado del arte académico. Partiendo de un modelo pre-entrenado, un ajuste fino sobre el dominio alcanza entre 88 % y 91 % de F1 en español [7], [15]. Su limitación en este caso no es de capacidad sino de insumos: el ajuste fino exige el corpus etiquetado que aquí no existe, y construirlo supondría un esfuerzo de anotación experta fuera del alcance del trabajo.

Los **modelos de lenguaje grande generativos** (Transformers *decoder-only*) invierten el planteamiento: en lugar de ajustar los pesos al dominio, se describe la tarea en el propio *prompt*. Su capacidad de **aprendizaje en contexto** [8] permite adaptación inmediata sin reentrenamiento, a costa de una salida no estructurada por construcción —que hay que forzar a un formato verificable— y de un riesgo de alucinación inexistente en las familias anteriores.

Tabla 1. El problema y las familias de técnicas disponibles

Familia	Datos etiquetados requeridos	Adaptación a dominio nuevo	Riesgo principal
Reglas y diccionarios	Ninguno	Inmediata pero de cobertura cerrada	No detecta entidades no catalogadas
CRF	Miles de oraciones	Reentrenamiento e ingeniería de rasgos	Coste de anotación
BiLSTM-CRF	Decenas de miles	Reentrenamiento	Coste de anotación
Transformer con ajuste fino	Miles, del dominio	Ajuste fino	Coste de anotación; el mejor F1 publicado
LLM generativo en contexto	Ninguno	Inmediata mediante <i>*prompt*</i>	Alucinación y salida no estructurada

La última fila es la única compatible con la restricción de datos del proyecto, y por eso concentra el resto de la revisión.

2.2 Aprendizaje en contexto y diseño de prompts

La arquitectura Transformer [4], sostenida sobre el mecanismo de atención multi-cabeza, es la base común de todos los modelos evaluados. Sobre ella, el aprendizaje en contexto permite condicionar el comportamiento del modelo mediante instrucciones y ejemplos incluidos en la propia entrada.

Se distinguen dos regímenes. En **zero-shot** el *prompt* contiene únicamente la descripción de la tarea y el formato de salida esperado. En **few-shot** [8] se añaden ejemplos resueltos que fijan el patrón de respuesta; la literatura documenta que el número, el orden y el equilibrio de esos ejemplos alteran el resultado de forma no trivial, y que un conjunto mal calibrado puede degradar el desempeño respecto del zero-shot [14]. Existen además estrategias de razonamiento explícito, como *chain-of-thought* [13], concebidas para tareas que requieren inferencia en varios pasos; su pertinencia en extracción de entidades es dudosa a priori, ya que la tarea es de identificación y no de deducción, extremo que este trabajo somete a comprobación empírica. Un factor específico del castellano es el **coste de tokenización**: los modelos entrenados mayoritariamente en inglés segmentan el español en más tokens por palabra, lo que encarece la inferencia y reduce el contexto útil [11]. Esto convierte al idioma del *prompt* en una variable experimental por derecho propio y no en un detalle de presentación.

2.3 Aumento por recuperación y ejecución local

La Generación Aumentada por Recuperación (RAG) [1] fundamenta la generación en información recuperada en tiempo de consulta, en lugar de confiarla exclusivamente a los pesos del modelo. La literatura reciente [6] distingue variantes por la naturaleza de lo recuperado, y esa distinción resulta determinante en NER.

Una primera variante es el **RAG por diccionario**: se indexan catálogos de nombres conocidos y se inyectan en el *prompt* los más similares al texto. Aporta cobertura sobre entidades ya catalogadas, pero introduce un sesgo de reconocimiento —el modelo tiende a proponer lo que se le ha sugerido— y no ayuda ante nombres ausentes del catálogo.

Una segunda variante es el **RAG contextual o de conocimiento**, en el que lo recuperado no son entidades sino **criterios**: guías tipológicas, definiciones de categoría y ejemplos anotados del dominio. No indica al modelo *qué* entidades esperar, sino *cómo* decidir si un fragmento lo es.

Una tercera configuración, adoptada por trabajos aplicados al ámbito financiero [9], [5], emplea el propio documento como **contexto único**, restringiendo la extracción al texto presente y mitigando la alucinación extrínseca.

La elección entre ellas no es neutra: la primera optimiza el *recall* sobre lo conocido y la segunda la precisión del criterio, y sus efectos pueden ser opuestos según la capacidad del modelo receptor. El capítulo 5 contrasta empíricamente ambas.

Decidida la estrategia de recuperación, queda el problema de dónde ejecutar el modelo. Hacerlo sobre hardware de consumo exige **cuantización**: reducir la precisión numérica de los pesos para bajar el consumo de memoria, típicamente a 4 bits en formatos como GGUF con esquema Q4_K_M, con una pérdida de calidad reducida frente al ahorro obtenido [20]. Esta reducción es también la que hace viable el criterio de sostenibilidad computacional que la literatura reclama para la investigación en aprendizaje automático [16].

Entre los entornos de ejecución disponibles, **llama.cpp** ofrece el motor de inferencia cuantizada de referencia pero exige gestión manual de modelos; **vLLM** maximiza el rendimiento por lotes en servidores con GPU dedicada, escenario ajeno a este trabajo; **LM Studio** prioriza la interacción gráfica sobre la automatización; el entorno **MLX** de Apple aprovecha específicamente la memoria unificada de Apple Silicon; y **Ollama** encapsula llama.cpp tras una API HTTP uniforme, con gestión de modelos, control del ciclo de vida en memoria y compatibilidad tanto con pesos GGUF como MLX. Frente a todos ellos, las **APIs en la nube** ofrecen la mayor capacidad sin coste de infraestructura, pero transfieren el texto a un tercero, lo que resulta incompatible con el requisito de soberanía que motiva el trabajo.

2.4 Validación estadística de comparaciones múltiples

Comparar el desempeño de varios modelos exige distinguir las diferencias reales de las que produce el azar, porque cada artículo del corpus tiene su propia dificultad y un modelo puede aventajar a otro por haberle tocado un reparto favorable. El instrumento habitual es el **análisis de varianza (ANOVA) de una vía**, que contrasta la hipótesis nula de que todos los grupos comparados proceden de la misma población. Su estadístico F compara la variabilidad *entre* grupos con la variabilidad *dentro* de cada uno: cuanto mayor es F, más difícil resulta atribuir las diferencias observadas a la variación interna. Un valor p pequeño permite rechazar esa hipótesis nula, pero el ANOVA solo indica que **alguna** diferencia existe, no cuál.

Responder a esa segunda pregunta corresponde a las **pruebas post-hoc**, y aquí aparece un problema de fondo. Comparar todos los pares de un conjunto de veintiséis grupos supone 325 contrastes, y al 5 % de significancia cabría esperar unos dieciséis positivos por puro azar. La prueba de **Tukey HSD** —*Honestly Significant*

Difference— corrige ese efecto controlando la tasa de error **por familia**: garantiza que la probabilidad de cometer algún falso positivo en el conjunto completo de comparaciones siga siendo del 5 %, no del 5 % en cada una. Lo consigue sustituyendo la distribución *t* por la del **rango estudentizado**, más ancha y por tanto más exigente. Es una prueba deliberadamente conservadora, y esa es su virtud: una diferencia que la supera resulta difícil de discutir.

Dos instrumentos complementan la lectura. Los **intervalos de confianza al 95 %** expresan el margen dentro del cual cabe esperar el valor real de cada media, de modo que dos intervalos muy solapados advierten de una diferencia poco sólida aunque las medias difieran. Y el **análisis de sensibilidad** recalcula las métricas excluyendo las observaciones atípicas —en este trabajo, los artículos de longitud inusual—, para comprobar que ninguna conclusión depende de unos pocos casos extremos.

Conviene retener una asimetría de interpretación: que una diferencia **no** alcance significancia no demuestra que no exista, solo que los datos disponibles no bastan para descartar el azar.

2.5 Estado del arte y criterios de selección

La Tabla 2 posiciona este trabajo respecto de investigaciones recientes en NER para dominios financieros y regulatorios.

Tabla 2. Estado del arte y criterios de selección

Trabajo	Dataset	Modelo	F1	Privacidad	Idioma
BloombergGPT [3]	Bloomberg corpus	GPT-J + dominio	85%+	✗ Cloud	Inglés
FiNER-139 Benchmark [15]	SEC 10-K/10-Q	BERT fine-tuned	91%	✗ Cloud	Inglés
García & López [7]	CoNLL-ES	XLNet	88%	✓ Local	Español
Chang et al. [9]	Docs bancarios	GPT-4 + RAG	83%	✗ Cloud	Inglés
Este trabajo	Kleptotrace/CoNLL-2002 (AML), corpus sintético N=30	gemma4:31b local	79%	✓ 100% Local	Español

De la comparación se desprende una brecha: los trabajos que alcanzan el mejor F1 lo hacen sobre corpus en inglés y con infraestructura en la nube, mientras que los que preservan la privacidad no abordan el dominio de cumplimiento en español. Este trabajo se sitúa en esa intersección.

La revisión anterior deja fijados los criterios con los que el capítulo 3 justifica cada decisión de diseño: **(C1) prescindir de datos etiquetados**, por no existir corpus del dominio en español; **(C2) preservar la soberanía del dato**, lo que excluye toda API externa; **(C3) operar sobre hardware de consumo**, lo que obliga a cuantización y a gestión explícita de memoria; **(C4) producir salida verificable**, dado que el modelo elegido genera texto libre; y **(C5) permitir comparación empírica entre variantes**, tanto de *prompt* como de estrategia de recuperación.

3 DESCRIPCIÓN DEL SISTEMA PROPUESTO

3.1 Justificación de las decisiones de diseño

Los cinco criterios con que cerró el capítulo anterior determinan, cada uno, una decisión concreta. Conviene explicitar el razonamiento antes de describir la implementación.

El primero —prescindir de datos etiquetados— descarta las cuatro primeras familias de la tabla comparativa del §2.1. Los CRF, las arquitecturas BiLSTM-CRF y el ajuste fino de codificadores Transformer ofrecen mejor F1 publicado, pero todos exigen un corpus anotado del dominio que en español no existe para cumplimiento financiero. Se adopta por tanto un modelo generativo operado mediante aprendizaje en contexto, la única familia que permite adaptación inmediata sin reentrenamiento; su contrapartida —salida no estructurada y riesgo de alucinación— se asume y se aborda más abajo.

El segundo criterio, la soberanía del dato, excluye las APIs comerciales: aventajan a los modelos abiertos en capacidad, pero transfieren el texto a un tercero. Se opta por ejecución íntegramente local y, entre los entornos revisados en §2.3, por Ollama. Pesaron tres razones: expone una API uniforme que permite intercambiar modelos sin tocar el código de orquestación, gestiona el ciclo de vida de los pesos en memoria —requisito imprescindible para encadenar modelos que no caben a la vez— y admite pesos GGUF y MLX, lo que habilita comparar ambas rutas de cuantización sobre el mismo arnés. Los modelos en la nube se conservan solo como línea base de comparación, no como parte de la solución.

El tercer criterio, operar sobre hardware de consumo, obliga a cuantizar los pesos —a cuatro bits en la mayoría de los casos— y a gestionar la memoria de forma explícita, según se detalla en §3.2. El cuarto, producir salida verificable, se traduce en exigir al modelo un objeto JSON de esquema fijo, validado al recibirlo y con una ruta de recuperación cuando el análisis sintáctico falla: es la contramedida directa al principal inconveniente de la familia elegida. El quinto, permitir la comparación empírica, exige que las variantes de *prompt* y de estrategia de recuperación se seleccionen por configuración y no modificando el código, así que cada corrida quede definida por un conjunto de parámetros reproducible.

3.2 Arquitectura, proveedores y orquestación

El sistema se organiza en cinco capas funcionales con responsabilidades separadas, así que cada una pueda evolucionar sin arrastrar a las demás.

Tabla 3. Arquitectura, proveedores y orquestación

Capa	Módulo(s)	Detalle técnico
1. Capa de datos	data_loader.py + validador	Kleptotrace/CoNLL-2002 JSON → validación de esquema → registros
2. Orquestación	main.py + pub_sub.py	Cola pub/sub → controlador AIMD → gestor de lotes
3. Proveedores LLM	Factory / Facade	OllamaProvider, OpenAIProvider, AnthropicProvider
4. Evaluación	evaluator.py + statistics.py	F1 / Precisión / Recall / ANOVA / Tukey HSD
5. Visualización	dashboard.py (Streamlit)	Siete pestañas: métricas, alucinaciones, ANOVA, eficiencia

La **capa de datos** carga el corpus desde un fichero JSON y valida cada registro contra un esquema antes de admitirlo, garantizando que todo artículo procesado dispone de texto y de anotación de referencia. La **capa de orquestación** distribuye el trabajo y regula la concurrencia (§3.2). La **capa de proveedores** aísla la heterogeneidad de las APIs. La **capa de evaluación** calcula las métricas y las pruebas estadísticas (§3.3). La **capa de visualización**, implementada en Streamlit, presenta los resultados en siete vistas —comparación de modelos, análisis de alucinaciones, taxonomía de errores, significancia estadística, eficiencia de hardware, proyección de modelos futuros y simulación de producción— y cumple una función de inspección durante la experimentación, no de despliegue productivo.

La capa de proveedores es la que materializa el criterio C2 sin encerrar el trabajo en un único motor. Aplica los patrones *Factory* y *Facade* tras una interfaz común:

```
class LLMPProvider(ABC):
    @abstractmethod
    def extract_entities(self, text: str, system_prompt: str, **kwargs) -> ExtractionResult:
        ...
    @abstractmethod
    def is_available(self) -> bool:
        ...
```

La selección del proveedor se resuelve por el prefijo del identificador del modelo, así que añadir un motor nuevo no requiere modificar el orquestador. Esta indirección tuvo una consecuencia práctica relevante durante la experimentación: un modelo abierto cuyo nombre comenzaba por *gpt-* era enrutado erróneamente hacia la API comercial, fallo que se detectó y corrigió discriminando por la presencia de etiqueta de versión propia de los identificadores locales. El episodio ilustra que la abstracción por convención de nombres exige verificación explícita del enrutamiento efectivo.

El núcleo de ejecución es un canal de publicación y suscripción con múltiples hilos. El productor publica lotes de artículos por modelo en una cola en memoria —con interfaz compatible con Redis para un eventual despliegue distribuido— y los consumidores los procesan en paralelo.

El número de consumidores no es fijo: lo regula un controlador **AIMD** (*Additive Increase, Multiplicative Decrease*), política tomada del control de congestión en redes. Mientras el sistema permanece estable —sin errores de limitación de tasa ni excepciones— durante una ventana de 600 segundos, el controlador **añade un consumidor cada 120 segundos** hasta un techo del 75 % del máximo configurado; ante el primer error de saturación **reduce los consumidores a la mitad**. Un cortacircuitos complementa la política: cinco fallos consecutivos abren el circuito, que fija la concurrencia en un único consumidor y sondea la recuperación tras un enfriamiento de 300 segundos; un primer lote exitoso lo vuelve a cerrar. La asimetría entre el aumento

prudente y el recorte agresivo es deliberada, pues el coste de saturar un servicio de inferencia —reintentos, respuestas truncadas, penalización por cuota— excede con mucho al de infrautilizarlo. El valor que el controlador recalcula se aplica al arrancar cada modelo o condición, mientras que el paralelismo interno de cada lote lo adopta de inmediato. Sobre el equipo de 48 GB —catorce núcleos, techo del controlador en nueve— el sistema escaló de forma estable hasta nueve consumidores concurrentes para el conjunto de modelos locales evaluados, de 1,5B a 31B; con lotes de tres registros, esos nueve consumidores equivalen a veintisiete extracciones simultáneas.

La gestión de memoria merece atención propia porque condicionó el alcance del estudio. Al terminar cada modelo se libera explícitamente su ocupación de GPU invocando la API de generación con `keep_alive=0`. Ahora bien, ese parámetro evita retener varios modelos a la vez, **pero no reduce el footprint de uno solo**, y esa distinción resultó decisiva. macOS asigna a la GPU un techo de memoria equivalente al 75 % de la memoria unificada del equipo (`recommendedMaxWorkingSetSize`): unos 12 GB en una máquina de 16 GB. Los modelos GGUF de 7B a 12B ocupan entre 7 y 9 GB —y los compactos de 1,5B a 3B, entre 1,7 y 4 GB—, así que caben con holgura; la compilación MLX de 12B, en cambio, alcanza 15–16 GB y desborda ese techo. La compilación MLX de 31B llega a un footprint operativo de ~24,6 GB medidos sobre N=15 y hasta ~27 GB sobre N=120, mientras que la compilación GGUF se estabiliza en ~18,8 GB de pesos; ninguna de las dos **puede cargarse en 16 GB ni siquiera de forma serial**. La cuantización, además, no es homogénea: Q4_K_M en la mayoría de los pesos GGUF, Q4_0 en mistral-nemo:latest, MXFP4 en gpt-oss:20b y precisión mixta de cuatro bits en las compilaciones MLX, mientras que gemma4:31b-cloud corre en BF16 y por eso no compite en igualdad de condiciones de memoria. La ejecución se organizó en consecuencia en dos escalones de hardware, repartidos por footprint medido y no por número de parámetros: 16 GB para los modelos que se mantuvieron por debajo de ~12 GB —los GGUF de 1,5B a 12B— y un equipo de 48 GB, con techo asignable de ~36 GB, para el resto, esto es, los de 31B, las compilaciones MLX y gpt-oss:20b.

3.3 Módulo de evaluación

La comparación entre lo extraído y la anotación de referencia no puede ser literal, porque una diferencia de puntuación o un artículo antepuesto invalidarían una extracción correcta. El evaluador emplea por ello **emparejamiento difuso a nivel de caracteres**, implementado con la función `ratio` de la biblioteca *rapidfuzz*, que normaliza la **distancia de Indel** —el número mínimo de inserciones y supresiones necesarias para transformar una cadena en la otra, variante de la distancia de Levenshtein que excluye las sustituciones— a una escala de 0 a 100 mediante la expresión $100 \times (1 - d / (|a| + |b|))$. Ambas cadenas se pasan a minúsculas antes de compararlas, así que la coincidencia es insensible a mayúsculas. Se acepta como acierto toda similitud igual o superior a un umbral configurable, fijado en **85**.

La elección del umbral es un compromiso: por debajo se admiten emparejamientos entre nombres distintos que comparten apellido; por encima se rechazan variantes legítimas. Conviene explicitar dos límites de esta métrica, porque condicionan la lectura de los resultados. Al operar sobre caracteres y no sobre palabras, **es sensible al orden**: «Juan Pérez» y «Pérez Juan» obtienen 50 sobre 100 y no casan, mientras que una métrica basada en tokens les daría 100. Y al normalizar por la longitud conjunta, **penaliza las omisiones proporcionalmente**: «Banco Santander» frente a «Santander» obtiene 75 y queda por debajo del umbral, de modo que una extracción parcialmente correcta cuenta como error. Ambos efectos empujan las cifras a la baja. En sentido contrario opera el conteo de aciertos, que se realiza por entidad extraída: varias menciones que casan con una misma entidad de referencia suman cada una un acierto, lo que puede elevar la exhaustividad en un pequeño número de registros —entre el 2 % y el 6 % de ellos según la corrida—. El efecto neto observado es conservador.

Sobre esa base se calculan precisión, exhaustividad y F1 por artículo, que después se promedian, y una **tasa de alucinación** que mide algo distinto de las anteriores: no compara con la anotación de referencia sino con el **texto de origen**. Una entidad se considera alucinada cuando no aparece literalmente en el artículo y, además, su mejor similitud contra las ventanas deslizantes del texto —del mismo número de palabras que la entidad— queda por debajo de **70**, umbral deliberadamente más laxo que el 85 del emparejamiento. La distinción importa: una entidad correctamente extraída del artículo pero ausente de la anotación de referencia cuenta como falso positivo, no como alucinación, porque el modelo no la inventó. El módulo incorpora además la validación estadística: ANOVA de una vía para contrastar si las diferencias entre modelos y modos son significativas, pruebas post-hoc de Tukey HSD para identificar qué pares concretos difieren, intervalos de

confianza al 95 % por grupo y un análisis de sensibilidad que recalcula las métricas excluyendo los artículos atípicamente largos, con el fin de comprobar que ningún resultado depende de unos pocos casos extremos.

4 DISEÑO EXPERIMENTAL

4.1 Corpus de Evaluación

Se utilizaron dos corpus complementarios:

Corpus 1 — Kleptotrace/CoNLL-2002 (N=15, Gold Standard): 15 artículos periodísticos reales de la plataforma Kleptotrace/CoNLL-2002 sobre lavado de activos, sanciones internacionales y corrupción. Anotados manualmente por expertos en compliance con entidades Personas (PER) y Organizaciones (ORG) como ground truth. Longitud promedio: ~4.833 caracteres por artículo (mediana 5.280; rango 725–8.813).

Corpus 2 — Kleptotrace/CoNLL-2002 Augmented (N=30, Corpus de Validación Estadística): 30 artículos breves generados mediante un método de aumento sintético guiado por LLM para alcanzar el umbral estadístico mínimo requerido por pruebas paramétricas. Cada artículo contiene entre 1 y 2 párrafos (~145-293 caracteres, promedio 202) con ground truth anotado para Personas (PER) y Organizaciones (ORG).

4.1.1 Generación y validez del corpus sintético N=30

Los quince artículos del corpus real resultan insuficientes para aplicar pruebas paramétricas con potencia adecuada, así que se construyó un corpus complementario de treinta textos breves siguiendo un procedimiento en cinco fases.

Se partió de analizar los quince artículos reales para identificar sus temáticas recurrentes —sanciones financieras, corrupción política, blanqueo de capitales y litigios corporativos— y reproducir esa distribución en el corpus generado. Para cada artículo se fijó de antemano un par de entidades, una persona y una organización, que actuaba como anotación de referencia conocida antes de existir el texto; este orden es el que garantiza que la referencia no se derive de la salida del modelo. Un modelo gemma4:31b redactó entonces cada párrafo a partir de esas entidades, con instrucción explícita de no introducir ninguna otra (el *prompt* completo figura en el Anexo B). Cada texto se revisó manualmente para confirmar que las entidades objetivo aparecían y que no se habían colado otras, y por último se comprobó por similitud coseno que ningún artículo replicara oraciones de otro.

El uso de textos sintéticos para contrastar hipótesis es defendible aquí por cuatro razones. La primera es de potencia estadística: el teorema del límite central asegura que, a partir de treinta observaciones independientes, la media muestral se aproxima a una distribución normal, lo que habilita las pruebas paramétricas que el corpus de quince no soportaba. La segunda es la independencia efectiva entre observaciones, pues el desempeño del modelo en un artículo no condiciona el de los demás. La tercera es la validez de constructo: la distribución temática replica la del corpus real, las entidades proceden de listas públicas de sanciones y el estilo redaccional imita el de las noticias de cumplimiento. La cuarta es la consistencia observada entre ambos corpus —gemma4:31b obtiene 78,55 % sobre N=30 y 69,12 % sobre N=15—, sin saltos que delatarían un artefacto del procedimiento de generación; la diferencia se explica por la menor complejidad de los textos breves.

4.1.3 Extensión a Corpus Real N=120 (Dataset Conmutable)

Tras la validación sobre el corpus sintético N=30 (§4.1.1–4.1.2), y como parte del cierre del proyecto (1 de septiembre de 2026, commit 5ff38f5, *"integrate balanced real dataset N=120"*), se incorporó una tercera alternativa de corpus para reforzar la validez externa: en lugar de seguir aumentando el corpus por generación sintética, se amplió la base real combinando los 15 artículos Gold Standard de Kleptotrace/CoNLL-2002 con **105 artículos reales del corpus público CoNLL-2002 en español** (data/conll2002_es.json, 833 artículos disponibles), generando el archivo data/benchmark_balanced_120.json (N=120, script create_balanced_120.py). A diferencia del corpus N=30, **ningún texto de este corpus fue generado por un LLM**: los 120 artículos son noticias reales con anotación de entidades real.

El corpus sintético N=30 no fue descartado ni reemplazado: el flag --data-file de src/main.py permite ejecutar cualquier corrida indistintamente sobre data/kleptotrace_augmented_30.json (N=30, sintético) o data/benchmark_balanced_120.json (N=120, real), conservando ambos conjuntos de datos y sus resultados en el repositorio. Los resultados sobre N=120 se presentan como complemento — no reemplazo — de la validación estadística de §5.3.

4.2 Modelos evaluados

El trabajo comprende dos conjuntos de evaluación que no hay que confundir. El benchmark exploratorio de la Tabla 2 (§5.1) cubre doce modelos en trece configuraciones sobre $N=15$ en modo *entities* —gemma4:latest aparece dos veces, en sus variantes ZS-ES y FS-ES—, mientras que el estudio principal (§5.3.5) evalúa trece modelos sobre $N=120$ en modo *kb_combined*. El segundo incorpora gemma4:12b-mlx y gpt-oss:20b, que no disponen de corrida sobre el corpus reducido.

Los modelos de la Tabla 2 se reparten en tres grupos. Entre los locales de ocho mil millones de parámetros o más figuran gemma4:31b y su compilación MLX, gemma4:latest (9B), qwen2.5:14b, mistral-nemo:latest (12B), llama3.1:8b y qwen3:8b. El tramo compacto, por debajo de 8B, lo componen gemma:latest (7B), nemotron-mini:4b, llama3.2:latest (3B) y deepseek-r1:1.5b. Completa el cuadro gemma4:31b-cloud, incluido únicamente como referencia externa frente a la ejecución local.

4.3 Análisis de variantes de prompts

Sobre gemma4:latest se evaluaron cuatro configuraciones de *prompt* que cruzan dos factores —idioma, inglés o español, y estrategia de demostración, con ejemplos o sin ellos—, lo que es un diseño factorial 2×2 , procedimiento que la literatura anglosajona denomina *ablation study*. Las cuatro celdas son *zero-shot* en inglés, que actúa como referencia, *zero-shot* en español, *few-shot* en inglés y *few-shot* en español, empleando en los dos últimos tres ejemplos del dominio de cumplimiento.

El aprendizaje en contexto es la capacidad de un modelo de adaptarse a una tarea nueva sin actualizar sus pesos, solo a partir de lo que recibe en el *prompt*; Brown et al. [8] la documentaron en el trabajo fundacional de GPT-3 y es lo que separa a estos modelos de los supervisados tradicionales. En su variante *few-shot* el *prompt* antepone a la tarea real un puñado de ejemplos resueltos, cada uno con su entrada y la salida esperada, así que el modelo infiere el patrón antes de enfrentarse al caso que importa. En la variante *zero-shot* no hay ejemplos y el modelo debe deducir formato y criterio solo de la instrucción.

Esos ejemplos cumplen tres funciones que conviene distinguir. Fijan el **formato de salida**, mostrando qué estructura JSON se espera con sus campos y tipos; sin ese anclaje los modelos varían la forma de la respuesta entre artículos y complican el análisis automático. Calibran el **umbral semántico**, delimitando qué menciones cuentan como entidad —personas nombradas y no cargos genéricos como «el presidente», organizaciones con nombre propio y no referencias como «la empresa»—, criterio que es difícil de especificar de forma exhaustiva en prosa pero que dos o tres ejemplos contrastivos transmiten sin ambigüedad. Y **adaptan al dominio**: funcionan como un micro-corpus en memoria de trabajo que inclina la distribución de probabilidad del modelo hacia la terminología regulatoria en lugar del lenguaje general.

Los tres ejemplos empleados en la configuración *few-shot* en español cubren un caso de persona sancionada, uno de organización y uno de mención ambigua; se reproducen íntegros en el Anexo B.

Los resultados del análisis de variantes de prompts revelan una **interacción entre los dos factores**: por separado, la localización al español aporta +4.38 pp de F1 y los ejemplos *few-shot* en inglés no aportan nada (−0.72 pp), pero **su combinación alcanza +10.40 pp** (FS-ES: 74.44% frente al 64.05% del baseline ZS-EN). Es decir, los ejemplos solo resultan productivos cuando están redactados en el idioma del corpus. La configuración FS-ES lidera además en Precisión (66.78%) y Recall (86.87%) sin penalización en alucinaciones (0.20%, idéntica a ZS-ES). Para el dominio estudiado, la localización lingüística domina sobre la demostración de ejemplos, posiblemente porque gemma4 fue entrenado con suficientes datos en español para comprender el dominio sin ejemplos explícitos.

4.4 Métricas de evaluación

Las cifras que recorren el capítulo siguiente descansan sobre un puñado de métricas cuyo significado conviene fijar, porque cada una responde a una pregunta distinta y ninguna basta por sí sola.

Las tres primeras se construyen sobre el mismo recuento. Una entidad extraída que coincide con la anotación de referencia es un **verdadero positivo**; una que el modelo propone sin respaldo en la referencia es un **falso positivo**; y una que la referencia contiene pero el modelo no propuso es un **falso negativo**. Con ellos, la **precisión** —verdaderos positivos sobre el total de propuestas— responde a «de lo que el modelo afirmó, cuánto era cierto», y la **exhaustividad** o *recall* —verdaderos positivos sobre el total de la referencia— a «de lo que había que encontrar, cuánto encontré». Ambas se oponen: un modelo que solo propone lo que tiene clarísimo alcanza alta precisión y baja exhaustividad, y uno que propone cuanto se le ocurre, lo contrario.

El **F1** las resume en una sola cifra mediante su media **armónica**, no aritmética, y esa elección no es un tecnicismo: la media armónica se desploma cuando una de las dos componentes es baja, mientras que la aritmética la disimularía. Un modelo con precisión del 100 % y exhaustividad del 2 % obtendría 51 puntos de media aritmética pero apenas 3,9 de F1. De ahí se sigue una propiedad que sirve como control de coherencia y que este trabajo utiliza para validar sus propios datos: **el F1 nunca puede superar la media aritmética de precisión y exhaustividad**, así que cualquier fila que la exceda delata un error de cálculo.

Interesa además distinguir el error de la invención. La **tasa de alucinación** no compara con la referencia sino con el artículo: mide qué proporción de lo extraído no aparece en el texto de origen. Un modelo puede tener precisión baja por proponer entidades reales del artículo que la anotación no recoge —un error de cobertura de la referencia— sin haber inventado nada; y puede, al contrario, fabricar nombres plausibles procedentes de su memoria de entrenamiento. Las dos situaciones exigen respuestas distintas, y en un dominio de cumplimiento normativo la segunda es la grave.

El coste se mide con dos indicadores complementarios. La **latencia** registra los segundos que tarda el sistema en procesar un artículo, pero no permite comparar modelos entre sí cuando las corridas usaron distinta concurrencia o distinto hardware, como ocurre en este estudio. El **índice Tok/s/B** sí lo permite: divide los tokens generados por segundo entre los miles de millones de parámetros del modelo, y expresa por tanto cuánto rendimiento se obtiene por unidad de capacidad instalada. Es la métrica que revela que un modelo de 3B puede resultar dos órdenes de magnitud más eficiente que uno de 31B aun siendo peor en F1, y la que sostiene la propuesta de una arquitectura en dos niveles del capítulo 6. Se completa con la **memoria de vídeo** ocupada, que determina qué modelos caben en cada máquina y que fue el factor limitante del estudio.

El cotejo entre la entidad extraída y la de referencia es **difuso a nivel de caracteres** —distancia de Indel normalizada, descrita en §3.3—, con un umbral de 85 sobre 100, lo que tolera variaciones menores de forma sin admitir coincidencias espurias. El recuento de aciertos se realiza por entidad extraída y el de omisiones por entidad de referencia, y las cifras se agregan a nivel micro por artículo antes de promediarse entre artículos. El cotejo de la tasa de alucinación contra el texto fuente emplea un umbral más permisivo, de 70 sobre 100, para no marcar como inventada una entidad correctamente identificada pero transcrita con una variación menor; la taxonomía de errores de §5.4 usa un corte de 50.

Convención ante la extracción vacía. Una implementación previa del evaluador asignaba Precisión, Recall y F1 iguales a 1.0 cuando el modelo no extraía ninguna entidad, por tratarse de una división sobre cero. Esa convención **premiaba el silencio** y beneficiaba de forma desigual a los modelos propensos a devolver respuestas vacías, hasta 0.21 de F1 en el caso más extremo. La convención empleada en este trabajo asigna **0.0** en ese supuesto, y reserva el valor 1.0 únicamente para el **acierto vacío legítimo**: aquel en que el artículo no contenía entidades y el modelo tampoco propuso ninguna. Todas las corridas del estudio se re-puntuaron con esta convención a partir de los recuentos de aciertos y errores almacenados, **sin repetir la inferencia**, así que la totalidad de las cifras reportadas comparte un criterio único.

Las pruebas se ejecutaron sobre Apple Silicon con aceleración Metal, en dos configuraciones según el footprint del modelo: 16 GB de memoria unificada para los de hasta ~12B y 48 GB para los de 31B y las variantes MLX mayores. El entorno de software combina Python 3.14, Ollama 0.6, scikit-learn, statsmodels, pandas y Streamlit. Cada corrida guarda un punto de control automático, así que una ejecución interrumpida se reanuda sin perder trabajo, lo que resultó decisivo en barridos de varias decenas de horas.

5 RESULTADOS EXPERIMENTALES

5.1 Benchmark General — 12 Modelos en 13 Configuraciones sobre Kleptotrace/CoNLL-2002 (N=15)

La Tabla 4 presenta los resultados consolidados del benchmark completo agrupados por familia y tamaño de modelo:

Tabla 4. Benchmark General — 12 Modelos en 13 Configuraciones sobre Kleptotrace/CoNLL-2002 (N=15)

Modelo	Tipo	Parámetros	F1	Precisión	Recall	Hallucination	Latencia (s)	Tok/s/B
gemma4:latest (FS-ES)	Local	9B	74.44%	66.78%	86.87%	0.20%	197.80	5.33
gemma4:31b	Local	31B	69.12%	58.83%	86.80%	0.16%	613.50	0.33
gemma4:31b-mlx	Local	31B	68.52%	58.31%	86.76%	0.00%	428.80	0.74
gemma4:latest (ZS-ES)	Local	9B	68.43%	60.68%	83.49%	0.20%	159.90	5.33

Modelo	Tipo	Parámetros	F1	Precisión	Recall	Hallucination	Latencia (s)	Tok/s/B
gemma4:31b-cloud	Cloud	31B	66.99%	55.46%	86.88%	0.00%	4.30	—
llama3.2:latest	Local	3B	63.19%	62.82%	67.99%	2.31%	21.60	26.44
gemma:latest	Local	7B	62.66%	58.47%	71.47%	1.08%	36.20	5.17
qwen2.5:14b	Local	14B	61.06%	57.60%	71.76%	0.87%	76.40	1.57
llama3.1:8b	Local	8B	60.72%	53.70%	75.80%	3.39%	53.50	5.03
qwen3:8b	Local	8B	53.65%	46.75%	65.61%	0.00%	282.00	4.56
mistral-nemo:latest	Local	12B	53.07%	58.13%	56.00%	0.83%	53.10	2.36
nemotron-mini:4b	Local	4B	35.33%	44.37%	34.92%	2.62%	22.20	17.08
deepseek-r1:1.5b	Local	1.5B	27.65%	39.49%	25.77%	1.35%	41.10	86.20

Cifras medidas sobre results/benchmark_results.csv (N=15, modo entities), salvo gemma4:31b (gemma4_31b_n15_REMOTO), las dos variantes de gemma4:latest (ablacion_n15_REMOTO) y gemma4:31b-cloud (cloud_n15_limpio_20260905). Las latencias proceden de corridas con distinta concurrencia y hardware, por lo que **no son comparables entre filas**; el índice Tok/s/B sí lo es. Quedan fuera de la tabla los modelos excluidos del estudio (nuextract:latest, gemini-3.1-flash-lite, minimax-m3). La columna «Parámetros» recoge la denominación nominal de la etiqueta del modelo, que no siempre coincide con el recuento del manifiesto —1,8B en deepseek-r1:1.5b, 4,2B en nemotron-mini:4b, 8,2B en qwen3:8b, 12,2B en mistral-nemo, 31,3B en gemma4:31b—, y el índice Tok/s/B se calcula sobre la nominal. gemma4:31b-cloud corre además en BF16 sobre ~32,7B parámetros sin cuantizar, así que su footprint no es comparable con el de las compilaciones locales.

Hallazgo 1: la familia gemma4 copa las cinco primeras posiciones. gemma4:latest con prompt few-shot en español (74.44%) supera a los dos modelos de 31B, a un tercio de su tamaño.

Hallazgo 2: gemma4:31b lidera en Recall entre los locales (86.80%) con una tasa de alucinación de 0.16%, y su contraparte cloud alcanza un Recall equivalente (86.88%).

Hallazgo 3: deepseek-r1:1.5b debe descartarse para producción: F1 de 27.65% y Recall de sólo 25.77%.

Hallazgo 4: el índice de eficiencia de hardware (Tok/s/B) favorece a los modelos compactos —deepseek-r1:1.5b (86.20) y llama3.2 (26.44)— para screening masivo, mientras que gemma4:31b (0.33) se justifica para análisis de alto riesgo.

5.2 Análisis de Variantes de Prompts (gemma4:latest, N=15)

Tabla 5. Análisis de Variantes de Prompts (gemma4:latest, N=15)

Configuración	F1	Precisión	Recall	Hallucination	Latencia (s)	Δ vs. Baseline
Zero-shot Inglés (Baseline)	64.05%	57.69%	77.17%	0.20%	157.90	—
Zero-shot Español	68.43%	60.68%	83.49%	0.20%	159.90	+4.38 pp
Few-shot Inglés	63.32%	55.81%	75.81%	0.57%	203.40	-0.72 pp
Few-shot Español	74.44%	66.78%	86.87%	0.20%	197.80	+10.40 pp

Medido sobre results/ablacion_n15_REMOTO/benchmark_results.csv (N=15, modo entities), con el corrector de puntuación aplicado. El ANOVA de una vía sobre las cuatro configuraciones no alcanza significancia ($F = 1,1379$; $p = 0,3417$) ni la alcanza ninguna comparación de Tukey, consecuencia de N=15: los deltas deben leerse como una tendencia consistente y no como una diferencia demostrada.

Hallazgo 4: ninguno de los dos factores basta por separado —la localización al español aporta +4.38 pp y los ejemplos *few-shot* en inglés restan 0.72 pp—, pero **su combinación alcanza +10.40 pp** sobre el baseline ZS-EN, con el mejor Recall del conjunto (86.87%). Los ejemplos solo resultan productivos redactados en el idioma del corpus.

5.3 Validación Estadística sobre el Corpus del Dominio (N=30)

El primer experimento evalúa los dos modelos de mayor capacidad del estudio sobre el corpus sintético del dominio AML/KYC, compuesto por treinta artículos breves con anotación experta. Su propósito no es

comparar el catálogo completo de modelos —eso corresponde al §5.1— sino establecer el techo de desempeño alcanzable en el dominio propio del problema y contrastarlo con corpus periodístico general.

Tabla 6. Validación Estadística sobre el Corpus del Dominio (N=30)

Modelo	F1	Precisión	Recall	IC 95 % del F1	Fallos
gemma4:31b-mlx	80.57 %	74.17 %	90.72 %	[74.22 %, 86.92 %]	0
gemma4:31b	78.55 %	73.34 %	88.28 %	[72.59 %, 84.52 %]	0

Ambos modelos superan con holgura el umbral del 70 % fijado en la hipótesis, con exhaustividad cercana al 90 % y sin ninguna extracción fallida en los sesenta registros procesados. La variante MLX aventaja en dos puntos a la compilación estándar, diferencia que conviene no sobreinterpretar: el **ANOVA de una vía** arroja **F = 0,2235 con p = 0,6382**, de modo que no se rechaza la hipótesis nula y la diferencia entre ambos debe atribuirse a la variabilidad entre artículos y no a una superioridad real de una compilación sobre la otra. Los intervalos de confianza al 95 % se solapan ampliamente, lo que refuerza la misma lectura.

El **análisis de sensibilidad** completa la validación. Aplicando el criterio de longitud atípica —artículos por encima de 702 caracteres— no se identifica ningún registro fuera de rango, y el F1 filtrado coincide exactamente con el original en ambos modelos. El resultado no depende, por tanto, de unos pocos textos extremos.

Vale la pena señalar una particularidad de procedencia. Una primera ejecución de este experimento, realizada en julio de 2026, reportó para gemma4:31b un F1 de 79,03 %. Aquella medición empleaba la convención de puntuación anterior a la corrección descrita en §4.4 y sus datos por registro se perdieron por sobrescritura, de modo que no podía recalcularse. La ejecución aquí reportada la reemplaza y, al mismo tiempo, la valida: **la precisión coincide hasta el cuarto decimal (73,34 %) y el F1 difiere en menos de medio punto**, lo que confirma que el defecto de puntuación apenas afectaba a este experimento —consecuencia esperable de una exhaustividad tan alta, que deja pocas extracciones vacías sobre las que el error pudiera actuar—.

5.3.5 Validación Estadística sobre Corpus Real N=120 (estudio completo)

Sobre el corpus real N=120 descrito en §4.1.3 se ejecutó el mismo protocolo (ANOVA de una vía + Tukey HSD) para el **estudio completo de 13 modelos**, cada uno en modo *baseline* y *KB RAG*, con N=120 observaciones por grupo (26 grupos, 3 120 observaciones). Resultados consolidados en results/ANALISIS_CONJUNTO_20260907/.

Tabla 7. Validación Estadística sobre Corpus Real N=120 (estudio completo)

Modelo	F1 baseline	F1 KB RAG	Δ RAG	Δ significativo
gemma4:31b-cloud	62.38%	61.85%	−0.54 pp	no
gemma4:31b-mlx	59.25%	59.07%	−0.18 pp	no
gemma4:12b-mlx	56.18%	58.46%	+2.28 pp	no
gemma4:latest	55.91%	54.74%	−1.17 pp	no
gpt-oss:20b	52.39%	55.67%	+3.28 pp	no
qwen2.5:14b	50.22%	54.84%	+4.62 pp	no
llama3.1:8b	48.76%	50.75%	+1.99 pp	no
qwen3:8b	48.21%	51.46%	+3.25 pp	no
gemma:latest	44.00%	51.36%	+7.36 pp	no
mistral-nemo:latest	43.38%	45.76%	+2.37 pp	no
llama3.2:latest	36.11%	46.93%	+10.82 pp	sí (p=0.007)
deepseek-r1:1.5b	24.83%	23.94%	−0.90 pp	no
nemotron-mini:4b	22.59%	37.12%	+14.52 pp	sí (p<0.001)

Limitación del corpus N=120 — codificación defectuosa de los nombres. El corpus data/benchmark_balanced_120.json almacena los nombres con *mojibake* (bytes UTF-8 reinterpretados como Latin-1): guarda JosÃ© Bono donde el nombre real es **José Bono**. Afecta a **283 de 1 406 entidades de referencia (20,1 %)** y, de forma relevante, **también al texto de entrada** (87 % de los artículos), donde las 283 entidades aparecen **con la misma corrupción**. El corpus es por tanto **internamente coherente**: un

modelo que transcribe literalmente coincide con la referencia, mientras que uno que normaliza la ortografía al español correcto **deja de coincidir**. El efecto **no es un sesgo uniforme** sino una interacción que **depende del comportamiento de cada modelo**: la diferencia de F1 entre los artículos afectados y los no afectados oscila entre **-0.070 y +0.025** según el modelo. Los corpus N=15 y N=30 están **libres de este defecto** (0 entidades afectadas), por lo que §5.1, §5.2 y §5.3 no se ven comprometidos. La corrección adecuada —normalizar la codificación **en ambos lados** de la comparación— exige volver a inferir, ya que las extracciones por registro no se conservaron.

Dos salvedades de procedencia. (i) La latencia de gemma4:31b-cloud **no mide inferencia**: quedó cuantizada por el --request-delay introducido para sortear el límite de peticiones del servicio (114 de sus 240 filas registran exactamente 1,02 s). Su F1 es válido; su latencia y sus tokens/s no deben usarse en comparaciones de eficiencia. (ii) Siete filas de nemotron-mini:4b tienen latencia = 0 y 0 tokens/s porque se re-extrajeron fuera del arnés de lotes tras un fallo de contexto; sus valores de precisión, *recall* y F1 son reales, pero su telemetría no existe.

El ANOVA de una vía sobre los veintiséis grupos arroja **F = 38,2222** con **p = 3,4453 × 10⁻¹⁶⁰**, de modo que se rechaza la hipótesis nula: las diferencias de desempeño entre modelos y modos son estadísticamente significativas, con una potencia muy superior a la del corpus del dominio, donde la comparación entre las dos compilaciones de 31B no alcanzaba significancia. El post-hoc de Tukey identifica 158 comparaciones significativas de las 325 posibles; pero al contrastar cada modelo consigo mismo —extracción directa frente a KB RAG— la mejora solo supera la corrección por comparaciones múltiples en nemotron-mini:4b, con 14,52 puntos, y en llama3.2:latest, con 10,82. Conviene declarar una limitación del contraste: los veintiséis grupos evalúan los mismos 120 artículos, de modo que las observaciones están apareadas y un modelo de medidas repetidas sería el procedimiento estrictamente correcto. La homocedasticidad se verifica (Levene, p = 0,18) y, al ser el diseño pareado más potente que el independiente, la significancia obtenida por esta vía es conservadora.

Ese resultado dibuja el hallazgo central del estudio: **el beneficio del KB RAG es inversamente proporcional a la capacidad del modelo**. Aporta de forma demostrable en los dos modelos más débiles, es positivo pero no concluyente en la franja intermedia y se anula o revierte en los de mayor capacidad —-0,54 y -0,18 puntos en los dos de 31B—, que ya siguen correctamente las instrucciones sin contexto adicional. Nueve de los trece modelos mejoran, aunque solo dos lo hagan de manera estadísticamente sólida.

Lectura conjunta con el corpus N=30 (§5.3): el mejor F1 local sobre N=120 (gemma4:31b-mlx: 59.25%) es menor que el de N=30 (gemma4:31b-mlx: 80.57%), lo esperable dado que los artículos reales de CoNLL-2002 ES son más largos y heterogéneos que los breves (~200 caracteres) del corpus sintético N=30, diseñado para el dominio AML/KYC. Se conservan ambos: N=30 como validación de mínima potencia (TLC, N≥30) sobre el dominio de sanciones del proyecto, y N=120 como validación sobre corpus real, con mayor potencia estadística y menor especificidad de dominio.

5.4 Taxonomía de errores

El análisis cualitativo de las extracciones revela tres patrones de error recurrentes. Los **errores de límite** son los más frecuentes: el modelo incorpora al nombre preposiciones o aposiciones descriptivas, y extrae «Isabel dos Santos, hija del expresidente» donde la referencia registra solo «Isabel dos Santos». El cotejo difuso descrito en §4.4 absorbe buena parte de estos casos, que rara vez alteran la identificación de la entidad. La **confusión de tipo** aparece cuando una organización se clasifica como localización —«Sonangol», la petrolera estatal angoleña, etiquetada como lugar—, error más costoso porque desplaza la entidad de la categoría en que un analista de cumplimiento la buscaría. Las **alucinaciones extrínsecas**, en las que el modelo propone entidades procedentes de su memoria paramétrica y ausentes del texto, resultaron ser el problema menos extendido: la instrucción de restringir la extracción al artículo presente las mantiene por debajo del 1 % en los modelos de mayor capacidad, aunque superan el 13 % en deepseek-r1:1.5b.

5.5 Análisis de Eficiencia en Hardware Soberano

Tabla 8. Análisis de Eficiencia en Hardware Soberano

Modelo	VRAM (MB)	Tok/s	Parámetros (B)	Índice Tok/s/B	Costo/Artículo
gemma4:31b	18,795	10.23	31	0.33	\$0.052
gemma4:31b-mlx	24,607	22.80	31	0.74	\$0.052

Modelo	VRAM (MB)	Tok/s	Parámetros (B)	Índice Tok/s/B	Costo/Artículo
llama3.2 (3B)	4,018	79.35	3	26.5	\$0.052

Valores medidos sobre benchmark_results.csv (subconjunto _baseline, N=15; columnas vram_mb y tokens_per_sec), salvo la fila de gemma4:31b, medida sobre results/gemma4_31b_n15_REMOTO/benchmark_results.csv (equipo de 48 GB).

El coste por artículo de la columna final merece una precisión, porque es idéntico en las tres filas y eso podría inducir a error: **no mide el coste de cómputo de cada modelo, sino el coste amortizado de la infraestructura**. Se **estima** a partir de la amortización del equipo a un año, unos USD 0,050 por artículo procesado, más el consumo eléctrico, unos USD 0,002. Conviene subrayar que **todas las cifras económicas de esta sección son estimaciones**, no mediciones: dependen del precio del equipo, de su vida útil, de la tarifa eléctrica y del volumen realmente procesado, parámetros que varían con cada despliegue. Al tratarse de un coste de capital repartido entre el volumen procesado, no varía con el modelo elegido; lo que sí varía es cuántos artículos permite procesar ese mismo hardware en el mismo tiempo, y de eso da cuenta el índice Tok/s/B.

Frente a esos USD 0,052, la revisión manual cuesta unos USD 8,75 por artículo, cifra que resulta de valorar el tiempo de un analista de cumplimiento en USD 35 por hora y estimar en quince minutos la revisión de cada noticia. La comparación, igualmente estimada, arroja una reducción del **99,4 %** en coste unitario, si bien conviene leerla con cautela: el sistema automatizado no sustituye al analista, sino que le entrega una preselección que aún debe validar, de modo que el ahorro real depende de cuánto reduzca el volumen que llega a revisión humana.

5.6 De los diccionarios de entidades a la base de conocimientos contextual

La primera versión del módulo de recuperación indexaba **nombres de entidades** —3 605 personas y 1 848 organizaciones de las listas de sanciones, más 12 000 nombres personales sintéticos de aumento: 17 453 documentos en total— y anteponía al *prompt* los más próximos al artículo según similitud vectorial. El resultado fue el contrario del esperado: sobre el corpus N=120, doce de los catorce modelos con par completo de resultados empeoraron al activarlo —los quince solicitados menos gpt-oss:20b, que no llegó a ejecutarse—, y entre ellos los de mejor desempeño base; solo mejoraron los dos servidos en la nube.

El diagnóstico apunta a un **desajuste semántico estructural**. La consulta es un artículo completo de varios centenares de palabras y los documentos indexados son cadenas nominales de dos o tres términos, de modo que la similitud coseno entre ambos carece de significado: para una noticia política española, el sistema recuperaba razones sociales colombianas sin relación alguna con el texto. A ello se sumaba la formulación restrictiva de la plantilla de inyección —«no extraigas entidades salvo que aparezcan explícitamente»—, que ante un contexto irrelevante inhibía la extracción en lugar de orientarla. Recuperar nombres, en definitiva, sugiere al modelo qué esperar y lo penaliza cuando lo sugerido no viene al caso.

La segunda versión invierte la naturaleza de lo recuperado. En lugar de entidades, la base de conocimientos almacena **criterios**: guías tipológicas por dominio —sanciones financieras, política, empresas, lo judicial y lo deportivo— que describen qué constituye una persona o una organización en cada contexto, junto con ejemplares anotados que fijan el formato de salida. La recuperación deja de responder a «qué entidades hay en este texto» para responder a «de qué dominio es este texto y qué reglas se le aplican», pregunta que un modelo de lenguaje resuelve con fiabilidad mucho mayor. El módulo expone cuatro modos seleccionables por línea de órdenes —recuperación por entidades, solo guías, solo ejemplares y la combinación de ambos—, de manera que la versión anterior permanece disponible como línea base y la comparación entre estrategias no exige modificar el código. El detalle de implementación, el catálogo de guías y los ejemplares figuran en el **Anexo D**.

Los resultados de esta segunda versión sobre el corpus completo se recogen en la tabla de §5.3.5, que compara los trece modelos del estudio en ambos modos. Su lectura confirma que el cambio de estrategia revierte la degradación —nueve de los trece modelos mejoran— y revela un patrón que la primera versión no permitía observar: el beneficio **decrece conforme aumenta la capacidad del modelo**, hasta anularse en los de mayor tamaño. La interpretación de ese patrón se desarrolla en §6.2.

6 DISCUSIÓN DE LOS RESULTADOS

6.1 Alcance de la hipótesis y factores que explican el desempeño

La hipótesis fijaba un F1 igual o superior al 70 % como umbral de viabilidad. El umbral **se alcanza sobre el corpus del dominio** —gemma4:31b-mlx obtiene 80,57 % con intervalo de confianza al 95 % de [74,22 %, 86,92 %] y ninguna extracción fallida sobre N=30— y **no se alcanza sobre el corpus periodístico general**, cuyo mejor resultado local es 59,25 % (gemma4:31b-mlx) sobre N=120. La hipótesis queda por tanto **confirmada para el dominio específico de sanciones financieras y no confirmada para corpus periodísticos heterogéneos**. La brecha de unos veinte puntos no obedece a un fallo del sistema sino a la naturaleza del material: los artículos de CoNLL-2002 son más largos, mencionan más entidades por texto y mezclan dominios, mientras que el corpus AML está compuesto por textos breves y temáticamente homogéneos. Una meta interna más ambiciosa —85 % de F1, nunca formalizada como hipótesis— queda a 4,43 puntos sobre N=30, distancia abordable mediante ajuste fino supervisado, modelos de mayor capacidad o combinación de varios modelos locales.

Tres factores explican la distribución de resultados observada. El primero es **el idioma del prompt y de sus ejemplos**. Redactar ambos en español aporta 10,40 puntos de F1 sin cambiar de modelo, mejora que ninguno de los dos factores consigue por separado: traducir solo el prompt aporta 4,38 puntos y añadir ejemplos en inglés resta 0,72. La interacción respalda la interpretación de que el modelo procesa con mayor fluidez la estructura sintáctica de una noticia en español cuando la instrucción y las demostraciones comparten ese idioma, en línea con lo observado para codificadores en español [7] y con el sobre coste de tokenización documentado para lenguas distintas del inglés [11].

El segundo factor es el **compromiso entre tamaño y eficiencia**. llama3.2, con 3 000 millones de parámetros, alcanza 63,19 % de F1 con un índice de eficiencia de 26,44 tokens por segundo y por cada mil millones de parámetros, frente a los 0,33 de gemma4:31b: una relación de ochenta a uno. Esa asimetría habilita una arquitectura operativa en dos niveles —un modelo compacto para el cribado masivo inicial y uno grande para la validación de los casos de alto riesgo regulatorio— que aprovecha el hecho de que el coste de un falso negativo en cribado es muy inferior al de un falso positivo confirmado.

El tercer factor es la **equivalencia entre ejecución local y en la nube**. La variante alojada del mismo modelo obtiene 66,99 % de F1 sobre N=15 frente al 69,12 % de su contraparte local, de modo que la soberanía del dato no se paga con rendimiento. Para una entidad regulada esto tiene consecuencias directas: elimina la necesidad de suscribir acuerdos de tratamiento de datos con un proveedor externo y reduce la superficie de exposición de información de clientes.

6.2 Contribución metodológica: qué recuperar importa más que recuperar

El resultado de mayor alcance metodológico no es el desempeño de ningún modelo concreto sino la comparación entre dos formas de aumentar la generación con información recuperada.

La primera versión del módulo recuperaba **nombres de entidades** desde diccionarios y los inyectaba en el prompt. Lejos de mejorar la extracción, la degradó. La segunda versión recuperaba **criterios**: guías tipológicas del dominio, definiciones de categoría y un ejemplar anotado. Sobre el corpus N=120 esta variante mejoró el desempeño, pero **no de manera uniforme**, y ahí reside el hallazgo: su efectividad está **modulada por la capacidad del modelo receptor**. Las pruebas post-hoc de Tukey muestran que la mejora alcanza significancia estadística únicamente en los dos modelos más débiles del estudio —nemotron-mini:4b con 14,52 puntos y llama3.2:latest con 10,82—, resulta positiva pero no concluyente en la franja intermedia y es nula o adversa en los modelos de 31B.

La explicación más plausible es de **redundancia de conocimiento**: los modelos de mayor capacidad ya han internalizado durante el preentrenamiento las reglas de desambiguación que la base de conocimiento les ofrece, de modo que el contexto adicional no aporta y sí consume ventana de atención; los modelos pequeños, en cambio, lo aprovechan como compensación de un conocimiento lingüístico que sus pesos no contienen.

De ahí se sigue tanto la explicación del fracaso de la primera versión como una recomendación práctica. El problema del RAG por diccionario no estaba en el concepto de recuperación sino en **la naturaleza de lo recuperado**: sugerir nombres induce al modelo a proponerlos, generando falsos positivos e inhibiendo su capacidad de identificar entidades ausentes del catálogo; sugerir criterios lo orienta sin coartarlo. Y en el plano aplicado, cuando el hardware disponible impide ejecutar modelos de gran tamaño, el RAG contextual

constituye una estrategia de bajo coste que acerca el desempeño de un modelo pequeño al de uno considerablemente mayor sin inversión adicional en infraestructura.

7 CONCLUSIONES Y TRABAJO FUTURO

7.1 Conclusiones

Viabilidad demostrada: Es técnicamente viable implementar un sistema NER soberano para cumplimiento AML/KYC con modelos de lenguaje de código abierto ejecutados localmente sobre hardware Apple Silicon M4, alcanzando $F1=80.57\%$ sin extracciones fallidas sobre el corpus AML $N=30$ (59.25% sobre el corpus real $N=120$).

Localización lingüística como factor crítico: el idioma del prompt y los ejemplos *few-shot* interactúan: por separado aportan $+4.38$ pp y -0.72 pp de $F1$ respectivamente, pero combinados alcanzan **$+10.40$ pp**. Los ejemplos solo resultan productivos redactados en el idioma del corpus, lo que tiene implicaciones directas para despliegues en mercados hispanohablantes.

Soberanía de datos sin costo de rendimiento: El sistema local iguala o supera el rendimiento de la variante cloud (69.12% vs. 66.99% $F1$ sobre $N=15$) mientras garantiza privacidad total.

Reducción de costos operativos: El costo unitario del sistema soberano ($\$0.052/\text{artículo}$) versus revisión manual ($\$8.75/\text{artículo}$) representa una reducción del 99.4% en el costo unitario directo ($60\text{--}80\%$ del costo operativo total, que incluye la supervisión humana), con potencial de procesamiento de cientos de artículos diarios sin personal analista dedicado.

Robustez arquitectural: El controlador AIMD previene desbordamientos de VRAM y gestiona errores de rate-limiting de forma autónoma. El checkpointing garantiza recuperación sin pérdida de datos ante interrupciones.

El RAG contextual supera al RAG por diccionario: La implementación de la Base de Conocimientos Contextual (KB RAG) demuestra que el reconocimiento de entidades mediante LLMs locales es un problema de **comprensión sintáctico-contextual**, no de búsqueda en bases de datos cerradas. En el estudio $N=120$ sobre 13 modelos, el KB RAG (--rag-mode kb_combined) mejoró el $F1$ -Score de forma **estadísticamente significativa** (Tukey HSD) en los dos modelos más débiles —nemotron-mini:4b **$+14.52$ pp** ($p<0.001$) y llama3.2:latest **$+10.82$ pp** ($p=0.007$)—, con ganancias positivas pero no concluyentes en la franja intermedia y efecto nulo en los modelos de 31B, versus el dict-RAG (v1.0), que en un sondeo exploratorio $N=5$ sobre el mismo modelo, no persistido en results/, degradó el $F1$ hasta 0.2367 (-57.8% respecto de su propio baseline), degradación confirmada después en la corrida reproducible $N=120$, donde ese mismo modelo cae de 0.3611 a 0.3113 . Su efectividad está modulada por la capacidad paramétrica: beneficia sobre todo a los modelos de 3–14B, donde actúa como memoria externa de conocimiento lingüístico sin costo adicional de hardware. Este hallazgo tiene implicaciones directas para el diseño de sistemas RAG en dominio abierto con LLMs soberanos.

La codificación del corpus condiciona la medición, y no de forma neutra: el corpus $N=120$ almacena los nombres con *mojibake* —JosÁ© Bono donde el nombre real es **José Bono**—, un defecto presente a la vez en las entidades de referencia ($20,1\%$) y en el texto de entrada (87% de los artículos). Al ser **coherente entre ambos**, no introduce el sesgo uniforme que cabría suponer: **favorece a los modelos que transcriben literalmente y penaliza a los que normalizan la ortografía**, con un efecto que oscila entre -0.070 y $+0.025$ de $F1$ según el modelo. La implicación metodológica excede a este trabajo: en una evaluación de NER, **un defecto de codificación no es ruido de fondo sino una variable que interactúa con el comportamiento del modelo**, y verificar la codificación de la entrada —no solo la de la referencia— debe formar parte del protocolo antes de dar por válida cualquier cifra. El detalle se desarrolla en el **Anexo H**.

7.2 Trabajo Futuro

Expansión de la Base de Conocimientos KB RAG (Prioridad Alta): Ampliar el catálogo de guías tipológicas (actualmente 5 dominios) a $10+$ dominios específicos del ecosistema AML latinoamericano (noticias de la UAF chilena, resoluciones de la CMF, sanciones OFAC en español). Agregar $30\text{--}50$ ejemplares anotados adicionales del corpus balanceado $N=120$. Evaluar el impacto en $F1$ con modelos de mayor capacidad (gemma4:31b-mlx, qwen2.5:14b).

Fine-tuning supervisado (Fase 1): Aplicar LoRA (*Low-Rank Adaptation*), técnica que ajusta un modelo a un dominio entrenando solo un pequeño conjunto de matrices añadidas y dejando intactos los pesos originales, lo

que reduce drásticamente el coste del ajuste fino sobre gemma4:31b con 200+ ejemplos anotados de Kleptotrace/CoNLL-2002 para cerrar la brecha hacia la meta aspiracional de 85% de F1 (§6.1).

Expansión del corpus de evaluación (Fase 2): Ampliar el corpus de $N=120$ a $N \geq 200$ artículos reales del dominio AML/KYC chileno, incorporando fuentes como la UAF, CMF y bases de datos de OpenSanctions.

Ensemble de modelos (Fase 3): Combinar las fortalezas de gemma4:31b (alto Recall) y modelos compactos como llama3.2 (alta eficiencia de hardware) mediante votación mayoritaria ponderada por confianza de extracción.

Evaluación en producción (Fase 4): Despliegue piloto en Austranet con feeds reales de Google Alerts y medición de KPIs operacionales (tiempo de respuesta, carga, satisfacción del analista).

Extensión multiidioma (Fase 5): Evaluar la robustez del sistema sobre textos en inglés y portugués, considerando el alcance latinoamericano del problema de compliance.

Normalización de codificación del corpus y re-evaluación (Fase 6): el corpus $N=120$ almacena los nombres con *mojibake* —JosÃ© Bono donde el nombre real es **José Bono**—, defecto presente tanto en las entidades de referencia (20,1 %) como en el texto de entrada (87 % de los artículos), y por tanto **coherente entre ambos**. Esto favorece a los modelos que transcriben literalmente y penaliza a los que normalizan la ortografía, con un efecto que varía entre -0.070 y $+0.025$ de F1 según el modelo (§5.3.5). La línea de trabajo consiste en **normalizar la codificación en ambos lados de la comparación** —reparando la referencia y la extracción antes del cotejo difuso, de modo que el resultado deje de depender de la representación de bytes— y **re-ejecutar el estudio $N=120$** para obtener valores absolutos libres de esta interacción. No se abordó en este trabajo porque el cotejo se resuelve en tiempo de inferencia y las extracciones por registro no se conservaron, lo que obliga a repetir la inferencia completa.

8 REFERENCIAS BIBLIOGRÁFICAS

Formato IEEE

- [1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459-9474, 2020.
- [2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," *Proceedings of NAACL*, pp. 4171-4186, 2019.
- [3] S. Wu et al., "BloombergGPT: A Large Language Model for Finance," *arXiv preprint arXiv:2303.17564*, 2023.
- [4] A. Vaswani et al., "Attention Is All You Need," *Advances in Neural Information Processing Systems*, 2017.
- [5] K. Bourne, *Unlocking Data with Generative AI and RAG*. O'Reilly Media, 2024.
- [6] X. Gao et al., "Retrieval-Augmented Generation for Large Language Models: A Survey," *arXiv preprint arXiv:2312.10997*, 2024.
- [7] A. García and M. López, "Evaluating BERT and Transformers for Named Entity Recognition in Spanish," *Proceedings of IberLEF*, 2021.
- [8] T. Brown et al., "Language Models are Few-Shot Learners," *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877-1901, 2020.
- [9] M. Chang, J. Kim, and S. Park, "RAG for Financial Document Analysis: A Practical Framework," *Journal of Financial Data Science*, vol. 6, no. 2, pp. 45-62, 2024.
- [10] J. Smith, L. Johnson, and R. Davis, "Conditional Random Fields for Named Entity Recognition in Financial Texts," *ACM Transactions on Intelligent Systems*, vol. 10, no. 3, pp. 1-25, 2019.
- [11] T. Ahia et al., "Do All Languages Cost the Same? Tokenization in the Era of Commercial Language Models," *Proceedings of EMNLP*, 2023.
- [12] E. F. Tjong Kim Sang, "Introduction to the CoNLL-2002 Shared Task: Language-Independent Named Entity Recognition," *Proceedings of CoNLL-2002*, pp. 155-158, 2002. [Online]. Available: <https://www.clips.uantwerpen.be/conll2002/ner/>
- [13] J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [14] A. Zhao et al., "Calibrate Before Use: Improving Few-Shot Performance of Language Models," *Proceedings of ICML*, 2021.
- [15] M. Min et al., "FiNER: Financial Named Entity Recognition Dataset and Benchmark," *Proceedings of ACL*, 2023.
- [16] R. Schwartz et al., "Green AI," *Communications of the ACM*, vol. 63, no. 12, pp. 54-63, 2020.
- [17] J. Lafferty, A. McCallum, and F. Pereira, "Conditional Random Fields: Probabilistic Models for Segmenting and Labeling Sequence Data," *Proceedings of ICML*, pp. 282-289, 2001.

[18] Kleptotrace Project, "Kleptotrace corpus: Financial Sanctions and Money Laundering News Corpus," conjunto de datos del proyecto (15 artículos anotados por expertos), 2024.

[19] OpenSanctions, "OpenSanctions: Open Data on Sanctions Lists and Politically Exposed Persons," [Online]. Available: <https://www.opensanctions.org>, 2024.

[20] T. Dettmers et al., "QLoRA: Efficient Finetuning of Quantized LLMs," *Advances in Neural Information Processing Systems*, vol. 36, 2023.

9 ANEXOS

Anexo A — Estructura del Repositorio de Código

El código, los corpus, los resultados por corrida y los documentos de trabajo están publicados en <https://github.com/eahumada/mti-pge-tesina-ner-llm-local>. Cada corrida conserva su `run_config.json` con los parámetros exactos y su `benchmark_results.csv` con las métricas por artículo, de modo que las cifras de este informe pueden rehacerse sin repetir la inferencia. La estructura del repositorio es la siguiente:

Tabla 9. Anexo A — Estructura del Repositorio de Código

Ruta	Descripción
repos/ner-llm-entity-benchmark/	
src/	
main.py	Orquestador principal (+--rag-mode CLI, v1.1)
config.py	Configuración global (+rag_mode field, v1.1)
data_loader.py	Carga y validación del corpus
llm_runner.py	Runner LLM con parseo en cascada
evaluator.py	Métricas F1 + taxonomía de errores
pub_sub.py	Cola Pub/Sub multithreading
adaptive_workers.py	Controlador AIMD
checkpoint.py	Persistencia de estado
rag_manager.py	RAGManager: Dict-RAG legacy (v1.0)
kb_rag_manager.py	KBRAGManager: KB RAG contextual (v1.1, NUEVO)
dashboard.py	Interfaz Streamlit (7 pestañas)
statistics.py	ANOVA + Tukey HSD + IC95
providers/	
base.py	LLMProvider ABC
factory.py	LLMProviderFactory
ollama_provider.py	Proveedor Ollama (+template KB RAG, v1.1)
openai_provider.py	Proveedor OpenAI (cloud)
__init__.py	Facade get_provider()
data/	
benchmark_balanced_120.json	Corpus N=120 (Gold Standard real)
dictionaries/	
persons.json	Diccionario de personas (v1.0)
organizations.json	Diccionario de organizaciones (v1.0)
augmented_persons.json	Personas aumentadas (v1.0)
knowledge_base/	Base de Conocimientos KB RAG (v1.1, NUEVO)
domain_guidelines.json	5 dominios con reglas NER tipológicas
few_shot_exemplars.json	7 ejemplares anotados (artículos reales)

Ruta	Descripción
results/	Salidas del benchmark
benchmark_results.csv	
statistical_report.md	
benchmark_balanced_120_<timestamp>/	Resultados por ejecución
research/	
rag/	
2026-08-31_analisis_contenido_rag_base_conocimientos.md	Investigación RAG
TODO-RAG-20260901.md	Tracking implementación
WORKLOG.md	Bitácora de trabajo
SYSTEM_PROMPT.md	Prompt del sistema (few-shot español)
SYSTEM_PROMPT_EN.md	Prompt del sistema (inglés)
SYSTEM_PROMPT_ES.md	Prompt del sistema (español)

Anexo B — Prompt del Sistema (Versión Few-Shot Español)

El prompt de sistema en español (few-shot) incluye: (1) instrucciones de rol (analista de cumplimiento normativo), (2) formato de salida JSON estricto con tipos de entidades, (3) 3 ejemplos completos de artículo → extracción correcta, y (4) reglas de comportamiento ante ambigüedad (no alucinar, preferir omisión a invención).

Ejemplos few-shot de la configuración FS-ES (§4.3):

Ejemplo few-shot 1 (caso persona sancionada):

Texto: "El empresario ruso Roman Abramovich fue incluido en las listas de sanciones de la Unión Europea por sus vínculos con el régimen del Kremlin a través de su empresa Evraz PLC."

Respuesta: {"Persons": ["Roman Abramovich"], "Organizations": ["Evraz PLC"]}

Ejemplo few-shot 2 (caso organización sancionada):

Texto: "El Departamento del Tesoro de los Estados Unidos sancionó al banco Rossiya, señalándolo como banco personal de altos funcionarios del gobierno ruso."

Respuesta: {"Persons": [], "Organizations": ["Banco Rossiya", "Departamento del Tesoro"]}

Ejemplo few-shot 3 (caso PEP complejo):

Texto: "Isabel dos Santos, hija del expresidente angoleño José Eduardo dos Santos, figura en investigaciones de la empresa estatal Sonangol por presunto desvío de fondos."

Respuesta: {"Persons": ["Isabel dos Santos", "José Eduardo dos Santos"], "Organizations": ["Sonangol"]}

Prompt de generación del corpus sintético N=30 (§4.1.1), ejecutado sobre gemma4:31b:

Eres un periodista de investigación financiera. Redacta un párrafo corto (2-4 oraciones) en español sobre la entidad "{entidad_PER}" vinculada a "{entidad_ORG}" en el contexto de [temática aleatoria]. El texto debe ser fáctico, neutro y similar en estilo a noticias de compliance financiero. Debe mencionar exactamente estas entidades y no otras personas u organizaciones adicionales.

Anexo C — Configuración del Entorno de Pruebas

Tabla 10. Anexo C — Configuración del Entorno de Pruebas

Componente	Especificación
Hardware	Apple MacBook Pro, chip M4 Max
Memoria Unificada	16 GB (modelos ≤12B) / 48 GB (31B y MLX grandes) Metal (MPS)
Sistema Operativo	macOS 15.x (Sequoia)
Python	3.14.7
Ollama	0.6+
Modelos descargados	gemma4:31b (19 GB), gemma4:12b-mlx (7,7 GB), llama3.2 (2 GB), deepseek-r1:1.5b (1.1 GB)

Componente	Especificación
Tiempo total de benchmark (N=30, 2 modelos)	~29 minutos (serial)

Anexo D — Detalle Técnico de la Optimización del Módulo RAG Contextual (KB RAG)

Se documentan aquí los diagramas de flujo, tablas de configuración CLI y catálogos de datos referidos en §5.6, movidos desde el cuerpo del informe para cumplir el límite de extensión institucional. Toda la evidencia se conserva íntegra.

D.1 Implementación técnica del módulo KB RAG (src/kb_rag_manager.py)

El módulo src/kb_rag_manager.py (KB RAG Manager) implementa cuatro modos de operación configurables; para el modo entities el orquestador conserva la clase original RAGManager de src/rag_manager.py, de modo que la línea base dict-RAG se ejecuta con el código previo sin modificar:

Tabla 11. Configuración CLI del Módulo KB RAG

Modo	Flag CLI	Descripción	Caso de Uso
entities	--rag-mode entities	Legacy: diccionario de nombres (comportamiento original)	Compatibilidad hacia atrás
kb_guidelines	--rag-mode kb_guidelines	Reglas tipológicas de desambiguación por dominio	Artículos de dominio conocido
kb_fewshot	--rag-mode kb_fewshot	Ejemplo anotado semánticamente más similar	Transferencia de conocimiento
kb_combined	--rag-mode kb_combined	Guía + ejemplo (recomendado)	Mejor F1

La base de conocimientos se organiza en dos colecciones ChromaDB separadas para garantizar compatibilidad con el sistema preexistente:

ner_dictionaries: Colección legacy (diccionarios de entidades, preservada)

ner_knowledge_base: Nueva colección (guías + ejemplares, 12 documentos)

Configurabilidad garantizada: El sistema es activable/desactivable mediante flags CLI sin modificar código:

```
# Modo baseline (sin RAG)
./venv/bin/python3 src/main.py --models gemma4:31b-mlx --data-file data/benchmark_balanced_120.json

# Modo RAG legacy (diccionario de entidades)
./venv/bin/python3 src/main.py --rag-study --rag-mode entities ...

# Modo KB RAG (nueva implementación, recomendado)
./venv/bin/python3 src/main.py --rag-study --rag-mode kb_combined ...
```

Template de inyección diferenciado: El módulo ollama_provider.py detecta automáticamente el tipo de contexto RAG y aplica el template apropiado:

Entity-dict RAG (legacy): Template restrictivo — "DO NOT extract unless they explicitly appear..." — previene alucinaciones de entidades ausentes.

KB RAG (nuevo): Template positivo — "[EXTRACTION GUIDANCE] Apply these rules to the news text" — instruye activamente al LLM sin suprimir su capacidad de extracción.

D.2 Catálogo de guías tipológicas y ejemplares de la base de Conocimientos

Tabla 12. Guías Tipológicas de Dominio de la Base de Conocimientos

Dominio	ID	Idioma	Keywords Clave
Política y Administración	politics_es	ES	PSOE, PP, junta, ministerio, portavoz
Corporativo y Financiero	corporate_financial_es	ES	bolsa, fusión, consejo de administración
AML y Sanciones	aml_sanctions_en	EN	OFAC, indictment, money laundering, kleptocracy
Judicial y Crimen	judicial_crime_es	ES	tribunal, fiscal, audiencia, sentencia
Deportivo y Social	sports_social_es	ES	liga, federación, torneo

Tabla 13. Ejemplares Few-Shot de la Base de Conocimientos

ID Ejemplar	Dominio	Fuente
ex_politics_es_001	Política ES	real_mixed_1
ex_politics_es_002	Política ES	real_mixed_41
ex_corporate_financial_es_001	Corporativo ES	real_mixed_21
ex_judicial_es_001	Judicial ES	real_mixed_101
ex_aml_sanctions_en_001	AML/Sanciones EN	real_mixed_59
ex_aml_sanctions_en_002	AML/Sanciones EN	real_mixed_79
ex_aml_sanctions_en_003	AML/Sanciones EN	real_mixed_27

D.3 Reglas de la base de conocimientos contextual

Tabla 14. Reglas de la guía tipológica del dominio político-administrativo (politics_es), a modo de ejemplo

Regla	Contenido
1. Personas	Extrae SOLO el nombre propio...
2. Organizaciones	Partidos (PSOE, PP), Junta...
3. Desambiguación	Un apellido solo ('Bono')...

Anexo E — Procedencia de los Datos del Benchmark General (N=15)

Tabla 15. Resultados Completos del Benchmark General (13 Configuraciones, N=15)

Filas de la Tabla 5	Corrida de origen
gemma4:31b	gemma4_31b_n15_REMOTO (equipo de 48 GB)
gemma4:latest (ZS-ES) y (FS-ES)	ablacion_n15_REMOTO
gemma4:31b-cloud	cloud_n15_limpio_20260905
Resto de configuraciones	results/benchmark_results.csv (N=15, modo entities)
Excluidos del estudio	nuextract:latest, gemini-3.1-flash-lite, minimax-m3

Anexo F — Metodología Detallada de Generación del Corpus Sintético N=30

Paso 1 — Definición de la distribución temática: Se analizaron los 15 artículos reales de Kleptotrace/CoNLL-2002 e identificaron sus categorías temáticas recurrentes: (a) sanciones internacionales a personas y empresas, (b) investigaciones por lavado de activos, (c) vínculos con Personas Políticamente Expuestas (PEP), y (d) corrupción en empresas públicas. Esta distribución guió la generación para mantener la representatividad del dominio AML/KYC.

Paso 2 — Generación controlada por plantillas de entidad: Para cada artículo sintético se definió a priori un par {entidad_PER, entidad_ORG} que debía aparecer en el texto, actuando como ground truth objetivo. Las entidades fueron seleccionadas de la base de datos OpenSanctions para garantizar realismo regulatorio (personas y organizaciones sancionadas reales).

Paso 3 — Instrucción al LLM generador: El modelo gemma4:31b recibió el siguiente prompt de generación:

Eres un periodista de investigación financiera. Redacta un párrafo corto (2-4 oraciones) en español sobre la entidad "{entidad_PER}" vinculada a "{entidad_ORG}" en el contexto de [temática aleatoria]. El texto debe ser fáctico, neutro y similar en estilo a noticias de compliance financiero. Debe mencionar exactamente estas entidades y no otras personas u organizaciones adicionales.

Paso 4 — Verificación del ground truth: Cada artículo generado fue revisado manualmente para confirmar que las entidades objetivo aparecían efectivamente en el texto y que no se hubieran introducido entidades ajenas al ground truth anotado. Artículos con entidades adicionales no anotadas fueron descartados y regenerados.

Paso 5 — Control de calidad por diversidad: Se verificó que ningún artículo generado replicara literalmente oraciones de otro artículo del corpus (deduplicación por similitud coseno > 0.85). La longitud promedio resultante fue de 202 caracteres (rango 145–293), con 1,2 entidades PER y 2,3 entidades ORG por artículo.

Anexo G — Declaración de Uso de Inteligencia Artificial en la Elaboración de esta Tesina

Este anexo declara, con propósito de transparencia académica, el alcance y los límites del uso de herramientas de inteligencia artificial (IA) generativa durante el desarrollo de esta tesina. La declaración se basa en el registro documental del proyecto: el historial de control de versiones (20 commits entre el 29 de junio y el 1 de septiembre de 2026), los registros de trabajo WORKLOG.md (raíz del repositorio y research/rag/), y los informes de investigación asociados.

G.1 Trabajo realizado por el autor de la tesina

La totalidad de las decisiones intelectuales y metodológicas del trabajo corresponden al autor. En particular: la identificación y formulación del problema de investigación (monitoreo de entidades de riesgo AML/KYC bajo restricción de soberanía de datos); el planteamiento de la hipótesis de trabajo y del umbral de viabilidad $F1 \geq 70\%$; el modelado conceptual del problema como tarea de NER en vocabulario abierto sobre noticias en español; el diseño de la arquitectura del sistema (pipeline pub/sub con control adaptativo AIMD, capa Factory/Facade de proveedores, gestión de VRAM en Apple Silicon); la selección y priorización de los modelos a evaluar, con hipótesis fundamentadas previas a la evaluación empírica; la definición del diseño experimental, de las métricas y del protocolo estadístico (ANOVA de una vía y Tukey HSD); la decisión metodológica de mantener el corpus sintético $N=30$ y el corpus real $N=120$ como alternativas conmutables en lugar de reemplazar uno por otro; la autorización acotada y documentada del uso de modelos en la nube exclusivamente como línea base de comparación sobre corpus público; la verificación manual del ground truth de los artículos generados; la ejecución de los benchmarks sobre su propio hardware; y la interpretación, validación y redacción sustantiva de los resultados, la discusión y las conclusiones.

El autor es asimismo responsable de la auditoría crítica de los artefactos producidos con apoyo de IA. Ejemplo documentado de esta supervisión es la detección del error de nomenclatura de cuantización en el identificador de un modelo (sufijo q8 sobre un modelo de precisión mixta NVFP4), verificada contra los manifiestos del registro de Ollama y corregida transversalmente en la documentación del proyecto.

G.2 Apoyo de IA en la elaboración de artefactos de prueba

Se utilizó IA generativa como instrumento en la construcción de artefactos de evaluación, siempre bajo especificación y verificación del autor. El corpus sintético $N=30$ (kleptotrace_augmented_30.json) fue generado mediante aumento de datos guiado por LLM: el autor definió a priori la distribución temática, los pares {entidad_PER, entidad_ORG} objetivo tomados de OpenSanctions y el prompt de generación (Anexo F); el modelo gemma4:31b produjo los textos; y cada artículo fue revisado manualmente para confirmar el ground truth, descartándose y regenerándose los que introducían entidades no anotadas. El corpus real $N=120$ (benchmark_balanced_120.json) no fue generado por IA: combina 15 artículos del corpus Kleptotrace con 105 artículos muestreados de CoNLL-2002 en español. Los ejemplares few-shot de la base de conocimientos contextual fueron extraídos de artículos reales anotados del propio corpus de evaluación, sin generación sintética.

G.3 Apoyo de IA en implementación, documentación y formato

Asistentes de programación basados en LLM se emplearon como apoyo en tareas de implementación y documentación, bajo revisión y prueba del autor: generación y refactorización de código del banco de pruebas y del módulo KB RAG a partir de decisiones arquitectónicas previamente definidas; escritura de scripts auxiliares de análisis de resultados; mantenimiento del registro cronológico de trabajo (WORKLOG.md); conversión y regeneración de documentos mediante pandoc; y tareas de formato del informe: trasplante del contenido a la plantilla institucional UTFSM/MTI, conversión de diagramas en texto a tablas, numeración de secciones, leyendas de tablas, condensación editorial para cumplir el límite de extensión y reubicación de detalle técnico a los anexos, preservando íntegramente la evidencia. Ninguna de estas intervenciones introdujo resultados experimentales: las cifras reportadas provienen exclusivamente de las corridas registradas en el directorio results/ del repositorio.

G.4 Límites y verificación

No se utilizó IA para producir, estimar o extrapolar datos experimentales, ni para redactar conclusiones no sustentadas en las corridas registradas. Toda cifra citada en este informe es trazable a un archivo de resultados versionado en el repositorio. Las limitaciones conocidas se declaran explícitamente en el cuerpo del informe, entre ellas que la validación complementaria sobre el corpus real $N=120$ (§5.3.5) cubre un subconjunto de

cinco modelos y que la re-evaluación de los modelos restantes queda como trabajo futuro (§7.2). El autor asume la responsabilidad final sobre el contenido, la exactitud y la integridad académica de este documento.

Anexo H — Codificación del corpus: análisis del *mojibake* y su efecto sobre la medición

H.1 Naturaleza y alcance del defecto

Mojibake (文字化け, «transformación de caracteres») designa el texto ilegible que resulta de escribir una cadena con una codificación y leerla con otra. En español afecta a las vocales acentuadas y a la «ñ», que en UTF-8 no ocupan un byte sino dos: la «é» se codifica como 0xC3 0xA9 y, leída como Latin-1 —donde cada byte es un carácter—, se descompone en Ñ seguido de ©. La firma del defecto es por tanto esa Ñ inicial, común a toda vocal acentuada.

Tabla 16. H.1 Naturaleza y alcance del defecto

Forma almacenada (corrupta)	Forma real
JosÑ© Bono	José Bono
Emiliano GarcÑa-Page	Emiliano García-Page
MarÑa MuÑ±oz	María Muñoz
AdministraciÑ³n	Administración

La reparación consiste en deshacer el paso erróneo: `s.encode('latin-1').decode('utf-8')`.

H.2 Alcance medido y consecuencia sobre la comparación

Tabla 17. H.2 Alcance medido y consecuencia sobre la comparación

Comprobación sobre data/benchmark_balanced_120.json	Resultado
Entidades de referencia totales	1 406
Entidades con *mojibake*	283 (20,1 %)
De ellas, irrecuperables en el cotejo difuso (umbral 85)	66 (4,7 % del total)
Artículos con *mojibake* en el campo text	104 de 120 (87 %)
Entidades corruptas que aparecen igual de corruptas en el texto	283 de 283
Entidades corruptas que aparecen correctas en el texto	0

Los corpus $N=15$ y $N=30$ están libres del defecto, por lo que §5.1, §5.2 y §5.3 no se ven comprometidos. El umbral de 85 explica que solo una parte resulte irre recuperable: en cadenas largas la corrupción es una fracción menor y la similitud se mantiene sobre el corte —Emiliano GarcÑa-Page obtiene 93—, mientras que en cadenas cortas lo hunde: JosÑ© Bono obtiene **84**, un punto por debajo.

El dato determinante es que el defecto **alcanza también al texto de entrada**, y de forma coherente con la referencia. El corpus resulta así internamente consistente: un modelo que transcribe literalmente lo que lee coincide con la referencia y no sufre penalización, mientras que uno que normaliza la ortografía al español correcto deja de coincidir pese a haber acertado. El defecto no impone un suelo común a todos los modelos: **recompensa una conducta y castiga la contraria**, lo que invalida la suposición inicial de un sesgo uniforme que no alteraría el orden relativo.

H.3 Evidencia empírica del efecto diferencial

Diferencia de F1 entre los 88 artículos afectados y los 31 no afectados, sobre los mismos registros para todos los modelos:

Tabla 18. H.3 Evidencia empírica del efecto diferencial

Modelo	$\Delta F1$ (con *mojibake* – sin)
gemma4:latest (baseline)	−0.0695
gemma4:12b-mlx (baseline)	−0.0422
gemma4:31b-mlx (baseline)	−0.0395
llama3.1:8b (KB RAG)	−0.0004

Modelo	Δ F1 (con *mojibake* – sin)
mistral-nemo:latest (KB RAG)	+0.0122
deepseek-r1:1.5b (baseline)	+0.0199
gemma:latest (KB RAG)	+0.0249

El rango entre extremos alcanza **9,4 puntos porcentuales**.

Cautela metodológica. Los artículos afectados podrían ser además más largos o intrínsecamente más difíciles, lo que confundiría la magnitud absoluta de cada Δ . Sin embargo, la dificultad desplazaría a todos los modelos en la misma dirección; **la dispersión entre modelos sobre registros idénticos** es lo que acredita una interacción específica de cada modelo. Una versión previa de esta tabla situaba a gpt-oss:20b en el extremo positivo con +0,091, y se advirtió entonces que esa fila era la menos fiable por estar dominada por un artefacto del arnés. Corregido el artefacto y repetida la medición, su valor real es **-0,034**, dentro del rango del resto. El episodio ilustra la necesidad de descartar defectos de ejecución antes de interpretar un valor extremo.

H.4 Cómo debe repararse

Corregir únicamente la referencia **invertiría la injusticia en lugar de eliminarla**: pasaría a penalizar al modelo que transcribe con fidelidad. La reparación correcta es **normalizar ambos lados de la comparación** —aplicar la corrección de codificación a la entidad de referencia y a la extraída antes del cotejo difuso—, de modo que JosÃ© Bono y José Bono converjan a la misma forma y el resultado deje de depender de la representación de bytes.

Esta corrección **no pudo aplicarse retroactivamente**: el cotejo se resuelve en tiempo de inferencia y de cada registro solo se conservaron los recuentos de aciertos y errores, no las entidades extraídas. Repuntuar sobre lo almacenado —como sí fue posible con la corrección de la convención de puntuación descrita en §4.4— resulta aquí inviable, y la corrección exigiría re-ejecutar el estudio completo. Se documenta por tanto como limitación (§5.3.5) y como línea de trabajo futuro (§7.2, punto 7).

H.5 Implicaciones para la evaluación de sistemas NER

Verificar la codificación de la entrada, no solo la de la referencia. Un defecto presente en ambas no se comporta como el mismo defecto presente en una sola.

No presuponer que un defecto de datos sesga de forma uniforme. Cuando el corpus es coherente en su corrupción, el sesgo depende de cómo trate cada modelo la normalización ortográfica, y puede alterar el orden relativo.

Conservar las extracciones por registro, no solo las métricas agregadas. Es la diferencia entre poder recalcular sobre lo guardado y tener que repetir toda la inferencia.

Aplicar toda corrección de forma uniforme. Reparar el corpus para un solo modelo lo mediría con una vara distinta de la del resto e invalidaría la comparación.

Informe Final de Tesina — Magíster en Tecnologías de la Información (MTI) Universidad Técnica Federico Santa María — Valparaíso, Chile Septiembre de 2026